Subject	Computer Science
Group	4
Title	Comparing proximal policy optimisation(PPO) and Q-learning algorithms involved in reinforcement learning of the CartPole environment.
Research Question	To what extent is PPO a more efficient method of reinforcement learning compared to Q-learning in the CartPole environment ?
Word Count	3969 Words

Introduction

Decision making and agency are no longer the exclusive domain of human beings. Many issues in today's world require computer agents to make multiple decisions and consider various factors to reach a desired outcome. Thus, writing an explicit algorithm for such situations is not practical. This is precisely why artificial intelligence(AI), and machine learning solutions are so prevalent. Using machine learning, computer systems need not rely on explicit instructions from the programmer, but rather make policies and decisions based on pattern analysis of data.

RL is a type of machine learning in which an intelligent system or an agent learns from its environment through interaction[1]. To evaluate the 2 algorithms in terms of effectiveness, cartpole, a well-established environment for evaluating machine learning algorithms is used. I will be using the OpenAI Gym cartpole environment which is an open-source library for testing machine learning algorithms[2]. Cartpole is a classic RL problem where a pole is attached to a cart, and the goal is to balance the pole by moving the cart left or right[3]. The cartpole environment provides a controlled environment for reproducible testing, making it ideal for comparing the performance and behaviour of different reinforcement learning algorithms.

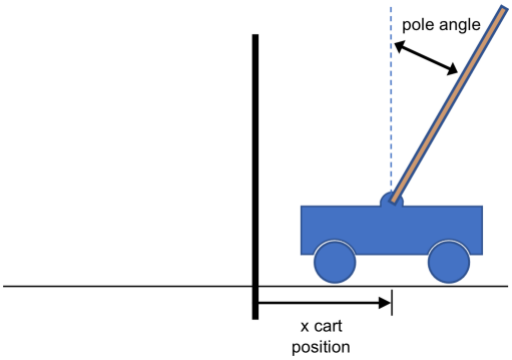

Figure 1. Cartpole environment example, a pole balanced on a truck[4].

[1] Brown, S. (2021, April 21). Machine learning, explained. MIT Sloan; MIT Sloan School of Management.
[2] Cart Pole - Gym Documentation. (n.d.). Www.gymlibrary.dev.
[3] *Introduction to RL and Deep Q Networks | TensorFlow Agents*. (2023). TensorFlow. https://www.tensorflow.org/agents/tutorials/0_intro_rl#:~:text=The%20Cartpole%20environment%20is%20one
[4] Galarnyk, M., & Mika, S. (2021, August 6). *Anyscale - An Introduction to Reinforcement Learning with OpenAI Gym, RLlib, and Google Colab*. Anyscale. https://www.anyscale.com/blog/an-introduction-to-reinforcement-learning-with-openai-gym-rllib-and-google

Within the realm of RL, Proximal Policy Optimization (PPO)[5] and Q-Learning[6] have emerged as pivotal algorithms[7], each with its unique approach to solving complex problems. With more algorithms being developed, the issue of efficiency comes to light. Thus, leading me to my research question:

To what extent is PPO a more efficient method of reinforcement learning compared to Q-learning in the CartPole environment ?

In this investigation, the PPO and Q-learning algorithms will be compared in the cartpole environment. The goal of RL is to maximize the reward of the agent by taking a series of actions in response to a changing environment[8]. This series of actions is called a policy, where an action is mapped to the state of the environment[9]. RL has applications in fields such as robotics, finance, and as a method of machine learning for AI, for decision-making in dynamic, complex environments. More and more companies are turning to RL to upgrade their computer systems, making the search for more efficient algorithms more relevant.

Background Information

Justification of algorithms chosen

PPO[10] and Q-learning are 2 distinct algorithms, making them ideal for comparison. Q-learning was first chosen in this investigation, as it is foundational in reinforcement learning, and is often used in simple environments. Since the simple CartPole environment was chosen, Q-learning would be optimal for solving this environment. PPO

[5] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Openai, O. (2017). Proximal Policy Optimization Algorithms.
[6] Kumar, S. (2020). Balancing a CartPole System with Reinforcement Learning -A Tutorial.
[7] DHMINE, O. (2023, November 23). Q-Learning and PPO: Driving Forces in OpenAI's AI Mastery. Medium.
[8] Taulli, T. (n.d.). Reinforcement Learning: The Next Big Thing For AI (Artificial Intelligence)? Forbes.
[9] Taulli, T. (n.d.). Reinforcement Learning: The Next Big Thing For AI (Artificial Intelligence)? Forbes.
[10] PhD, W. van H. (2023, January 31). Proximal Policy Optimization (PPO) Explained. Medium.

was subsequently chosen to complement Q-learning in the investigation, as they have multiple key points of difference such as their approaches to exploration[11] and exploitation[12] [13], action spaces[14] [15] and structure.

Description of cart pole environment

The CartPole environment is an OpenAI Gym environment[16]. OpenAI Gym is a library that is a toolkit for developing and comparing RL algorithms in a wide range of environments[17]. In the Cartpole environment a cart is placed on a frictionless track, with a pole mounted vertically on it as seen in Figure 2. The aim is to balance the pole upright as long as possible by applying either a left or right force to the cart[18]. The environment provides feedback in the form of rewards: a positive reward for each time period the pole stays upright and a termination signal when the pole deviates beyond a certain angle. The parameters in this environment are seen in the Table 1 below,

Parameter	Description
cart velocity(x)	Cart velocity refers to how fast the cart is moving.
cart position($\dot{x}$)	Cart position refers to the position of the cart with respect to its starting point.
angle of rotation of the pole(θ)	The angle of rotation of the pole refers to the angle at which the pole is tilting.
angular velocity($\dot{\theta}$)	Angular velocity refers to the rate at which the pole attached to the cart is rotating about its pivot

Table 1. Cartpole environment parameters

[11] Exploration is where the agent chooses actions randomly to explore possible actions and rewards.
[12] Exploitation is where the agent chooses actions from past experiences, with the specific goal of increasing rewards.
[13] The Exploration/Exploitation trade-off - Hugging Face Deep RL Course. (n.d.). Huggingface.co.
[14] Kumar, S. (2020). Balancing a CartPole System with Reinforcement Learning -A Tutorial.
[15] Proximal Policy Optimization — Spinning Up documentation. (n.d.). Spinningup.openai.com.
[16] Cart Pole - Gym Documentation. (n.d.). Www.gymlibrary.dev.
[17] Cart Pole - Gym Documentation. (n.d.). Www.gymlibrary.dev.
[18] Cart Pole - Gym Documentation. (n.d.). Www.gymlibrary.dev.

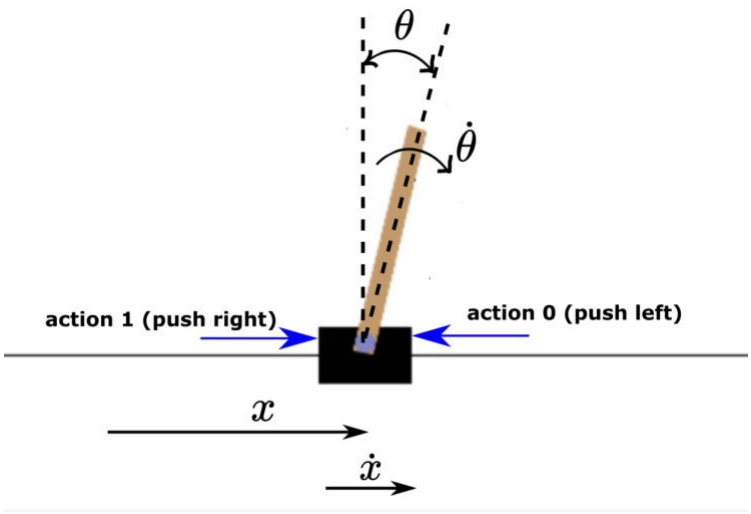

Figure 2. Cartpole environment visualization. Haber, A. (2023, January 21)[19]

The initial states at the beginning of each episode or trial are chosen randomly to be in the range (-0.1, 0.1) as starting states should deviate only slightly from the equilibrium positions, to allow the agent to learn how to balance it at various starting positions.

A reward of 1 is added each time the pole is balanced for a unit time in the episode. Since the goal of the algorithm is to maximize the reward of the intelligent system, the system will be trying to maximize the number of steps taken per episode[20]. In other words, it will try to balance the pole on the cart for a longer period, implying that *that it seeks to learn an effective policy for maintaining stability and prolonging the duration of successful pole balancing.*

[19] Haber, A. (2023b, January 31). *Detailed Explanation and Python Implementation of the Q-Learning Algorithm with Tests in Cart Pole OpenAI Gym Environment – Reinforcement Learning Tutorial – Fusion of Engineering, Control, Coding, Machine Learning, and Science*
[20] Cart Pole - Gym Documentation. (n.d.). Www.gymlibrary.dev.

Q-learning Algorithm

Q-learning is a value-based algorithm, which estimates the value of taking specific actions in particular states[21]. Figure 3 below illustrates how the Q-learning algorithm works in each time step of an episode.

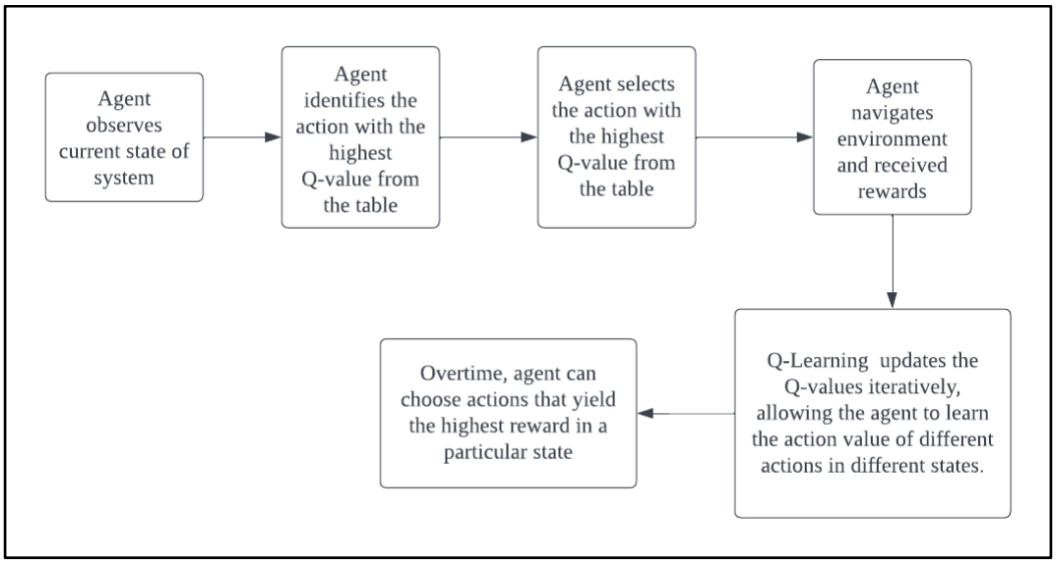

Figure 3. Q-learning algorithm process

[21] Shyalika, C. (2021, July 13). *A Beginners Guide to Q-Learning*. Medium. https://towardsdatascience.com/a-beginners-guide-to-q-learning c3e2a30a653c#:~:text=Q%2Dlearning%20is%20a%20values

For example, after an agent performs a few episodes in the environment, the table of Q values[22] for each action-state value might be as seen in Table 2[23].

State($x, \underline{\dot{x}}, \theta, \underline{\dot{\theta}}$)	Action 0 (Push Left)	Action 1 (Push Right)
(0, 0, 0, 1)	1.25	1.75
(0, 0, 1, 1)	0.39	0.87
(0, 0, 2, 1)	1.32	0.98
. . .	. . .	. . .

Table 2. Q-table for cart-pole environment

[22] A Q-table is a data structure used in RI to store and update the quality of actions, helping an agent make decisions based on the value of each state-action pair.
[23] Doshi, K. (2021, February 14). *Reinforcement Learning Explained Visually (Part 4): Q Learning, step-by-step*. Medium. https://towardsdatascience.com/reinforcement-learning-explained-visually-part-4-q-learning-step-by-step-b65efb731d3e

The Q-learning algorithm is updated using the Bellman equation, as seen in Figure 4[24]. It describes how the value of a state is related to the values of its preceding states. The Bellman equation is a recursive function that updates the Q-values iteratively as explained in Table 3.

$$Q(s, a) = Q(s, a) + \alpha * (r + \gamma * \max[Q(s', a')] - Q(s, a))$$

where,

Figure 4. Bellman Equation[25]

Variable	Description
Q(s, a)	Q-value of the action-state pair (s, a). It represents the expected reward when acting in state, s[26].
α	Learning rate, which I set to 0.1 per convention. It determines how much the latest action affects the new Q-value. As α increases, the magnitude of change in Q-value increases[27]
γ	Discount rate, which determines how much potential future rewards affect the new Q-value. As γ decreases, the influence of immediate rewards on the Q-value increases[28]
r	The reward received after acting in state, s.
max[Q(s', a')]	The maximum Q-value among all possible actions a' in the next state, s'. Represents the optimal Q-value in the next state[29].

Table 3. Q-learning algorithm variable

[24] Kumar, S. (2020). Balancing a CartPole System with Reinforcement Learning -A Tutorial.
[25] Kumar, S. (2020). Balancing a CartPole System with Reinforcement Learning -A Tutorial.
[26] Shyalika, C. (2021, July 13). *A Beginners Guide to Q-Learning*. Medium.
[27] Kumar, S. (2020). Balancing a CartPole System with Reinforcement Learning -A Tutorial.
[28] Sutton, R., & Barto, A. (n.d.). Reinforcement Learning An Introduction second edition.
[29] Shyalika, C. (2021, July 13). *A Beginners Guide to Q-Learning*. Medium.

```
Initialise  Q(s_n, a_n) arbitrarily
Initialise s_0 randomly in a range;

For n in range of episode_number:
        // random exploration for first x episodes
        If x =< 100
                Loop till s_t is terminal
                        Select action a randomly
                        Observe reward r and next state s_t+1
                        Update Q(s,a)
                        //Set state to next state
                        s_t = s_t+1

                End Loop
        // Epsilon greedy approach for remaining episodes to select
        actions
        For n in range of x to n:
                Loop till St is terminal
                        Select action a using epsilon greedy algorithm
                        Observe reward r and next state s_t+1
                        Update Q(s,a) using Bellman equation

                        //Set state to next state
                        s_t = s_t+1
                End Loop
```

Table 4. Q-learning pseudocode

The function Q(s,a) takes the state and the action as inputs and uses the epsilon greedy approach to optimise the policy[30]. In the epsilon greedy approach, the epsilon parameter, $e < 1$ is selected. For this investigation, I have made $e = 0.2$ in accordance with literature[31]. A random number, $r < 1$ is then chosen. If $r < e$, then a random action, 0 or 1 is chosen. If $r > e$, the greedy approach is used. In the greedy approach, the action that results in the highest value of the action-value function, Q(s,a), is chosen[32].

For example, for a state 3, Q(3, a) = [1.25, 1.75] where 1.25 and 1.75 denote the value of Q(3,a) for action 0 and 1 respectively. Since 1.75 > 1.25, the action 1 is selected. Figure 5 below shows the programme flowchart for the epsilon greedy algorithm.

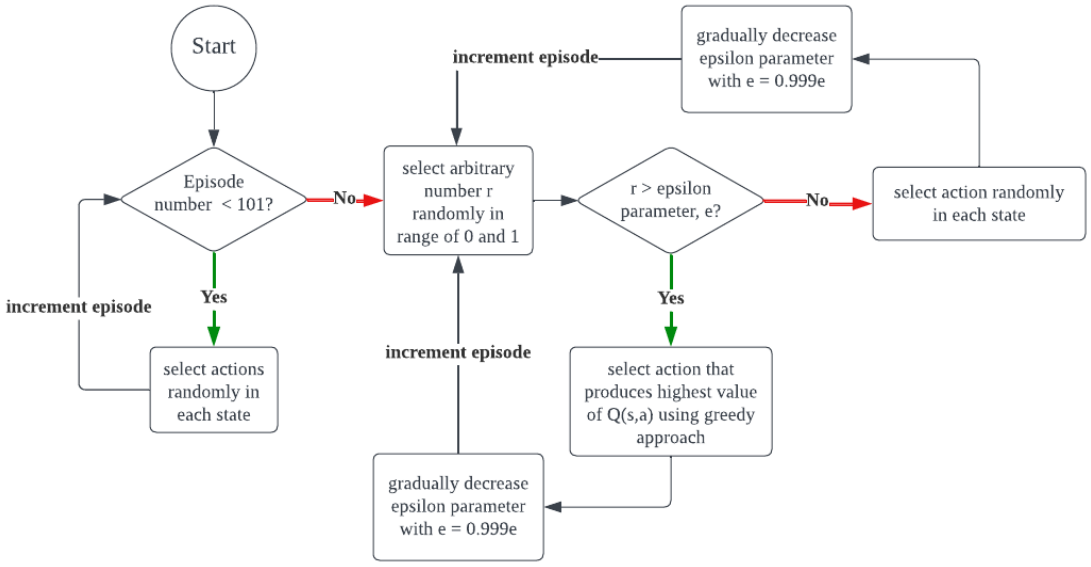

Figure 5. Epsilon -greedy programme flowchart

The pseudocode for the epsilon greedy approach is seen in Table 5.

[30] baeldung. (2020, December 18). Epsilon-Greedy Q-learning | Baeldung on Computer Science. Www.baeldung.com.
[31] Kumar, S. (2020). Balancing a CartPole System with Reinforcement Learning -A Tutorial.
[32] baeldung. (2020, December 18). Epsilon-Greedy Q-learning | Baeldung on Computer Science. Www.baeldung.com.

```
Initialise Q(s,a) arbitrarily

     Loop till s_n is terminal
          Select number r randomly in range of 0 to 1
          If r<e
               Select action 1 or 0 randomly
          If r>e
               When Q(s_n,s_n) = [x, y]

                    If x>y
                         Choose action 0
                    Is y>x
                         Choose action 1
     End loop
```

Table 5. Epsilon-greedy pseudocode

During the initial stages of Q-learning, the agent has no knowledge about the environment. If the system were to exploit its current knowledge by always choosing the action with the maximum Q-value using the greedy method, it would miss out on actions that may lead to higher rewards. Thus, exploration is introduced in the form of random choices to encourage the system to take random actions and explore the environment, so the system can gather more information about the rewards associated with different state-action pairs and discover better actions.

When comparing Figure 6 and 7, for a longer period, the agent performs better and reaps higher rewards having gone through exploration. Figure 6 shows the rewards obtained without exploration, where the system averaged rewards much less than 100 even towards the 1000th episode as seen by the red line. Figure 7 shows the rewards obtained with exploration, where the rewards gained increased and averaged closer to 100 as seen by the red line. Thus, after experimenting with the effect of exploration on rewards, I implemented exploration in the Q-learning algorithm, to maximize rewards for more episodes, and to better compare with PPO.

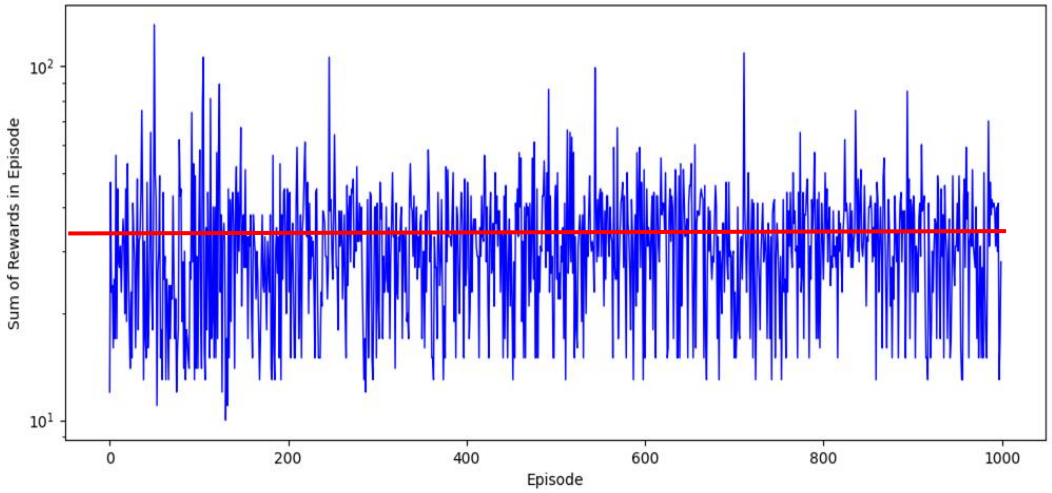

Figure 6. Rewards obtained with no exploration in Q learning.

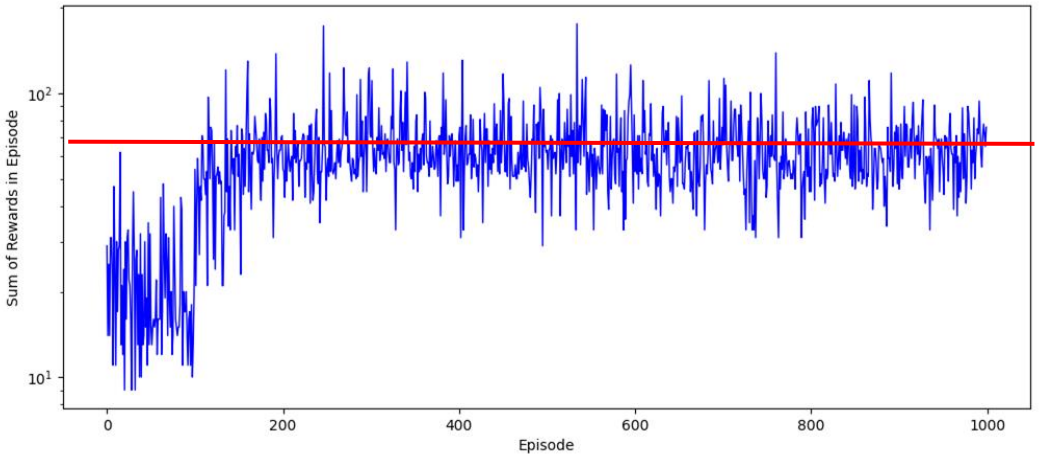

Figure 7. Rewards obtained with exploration in Q-learning.

Video 1. Self-generated experimental Q-learning video simulation for 200 Episodes
https://drive.google.com/file/d/1Dxp9LAm-x5ddjZSkSYa_U2gUbFTOEi-H/view?usp=sharing

Proximal Policy Optimisation (PPO)

PPO, unlike Q-learning, is a policy-based function. PPO works by updating an existing policy and ensures that the update is not too large, to ensure the new policy does not differ from the old policy too much[33]. This creates the optimal policy for the cartpole environment, as smaller updates are more likely to eventually provide the optimal policy[34].

The policy function of PPO works by outputting a probability distribution over the actions, and the agent tests from this distribution to make a choice of action[35]. The primary goal of the policy function is to maximize the reward received, in other words balancing the pole for a longer time.

PPO uses an actor-critic neural network[36]. There are 2 neural networks used for the actor and critic respectively. The actor is responsible for learning and selecting actions. The Critic network evaluates the actions suggested by the actor, to see how good or bad they are in a state[37].

[33] *A Brief Introduction to Proximal Policy Optimization*. (2021, September 1). GeeksforGeeks. https://www.geeksforgeeks.org/a-brief-introduction-to-proximal-policy-optimization/#:~:text=The%20most%20common%20implementation%20of

[34] Santhosh, S. (2023, January 2). Reinforcement Learning (Part-8): Proximal Policy Optimization(PPO) for trading environment(TensorFlow) Medium.

[35] DhanushKumar. (2024, February 21). *PPO Algorithm - DhanushKumar - Medium*. Medium; Medium. https://medium.com/@danushidk507/ppo-algorithm-3b33195de14a

[36] Santhosh, S. (2023, January 2). Reinforcement Learning (Part-8): Proximal Policy Optimization(PPO) for trading environment(TensorFlow) Medium.

[37] Karunakaran, D. (2020, September 30). *The actor-Critic Reinforcement Learning algorithm*. Intro to Artificial Intelligence. https://medium.com/intro-to-artificial-intelligence/the-actor-critic-reinforcement-learning-algorithm-c8095a655c14

For the Actor network, it takes in 4 input variables, the cart velocity, cart position, angle of rotation of the pole and angular velocity of the pole, like Q-learning. It then outputs the probability of taking each action, which is known as the policy function as seen below in Figure 8.

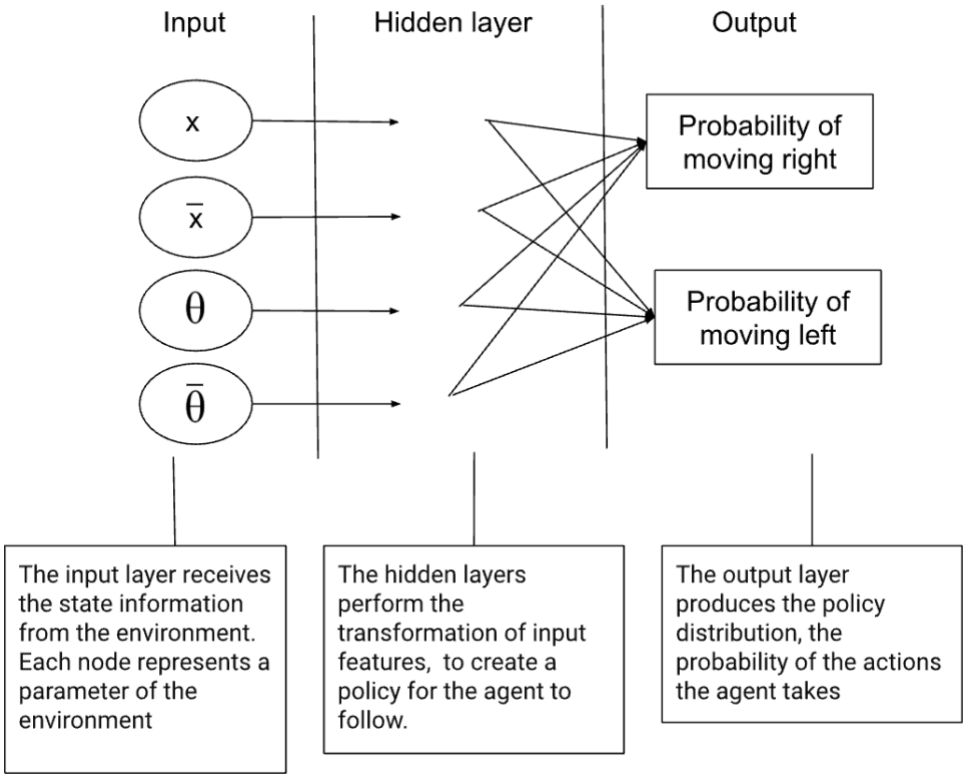

Figure 8. Actor neural network for PPO

The Critic network, as seen in Figure 9, also takes in cart position, cart velocity, angular velocity of the pole, and angle of rotation of the pole. However, it outputs the state-value function, which essentially how good the action is in terms of rewards obtained. The Critic network evaluates the actions suggested by the actor. The state-value function estimates the expected rewards of being in a certain state and following the actor network's action[38].

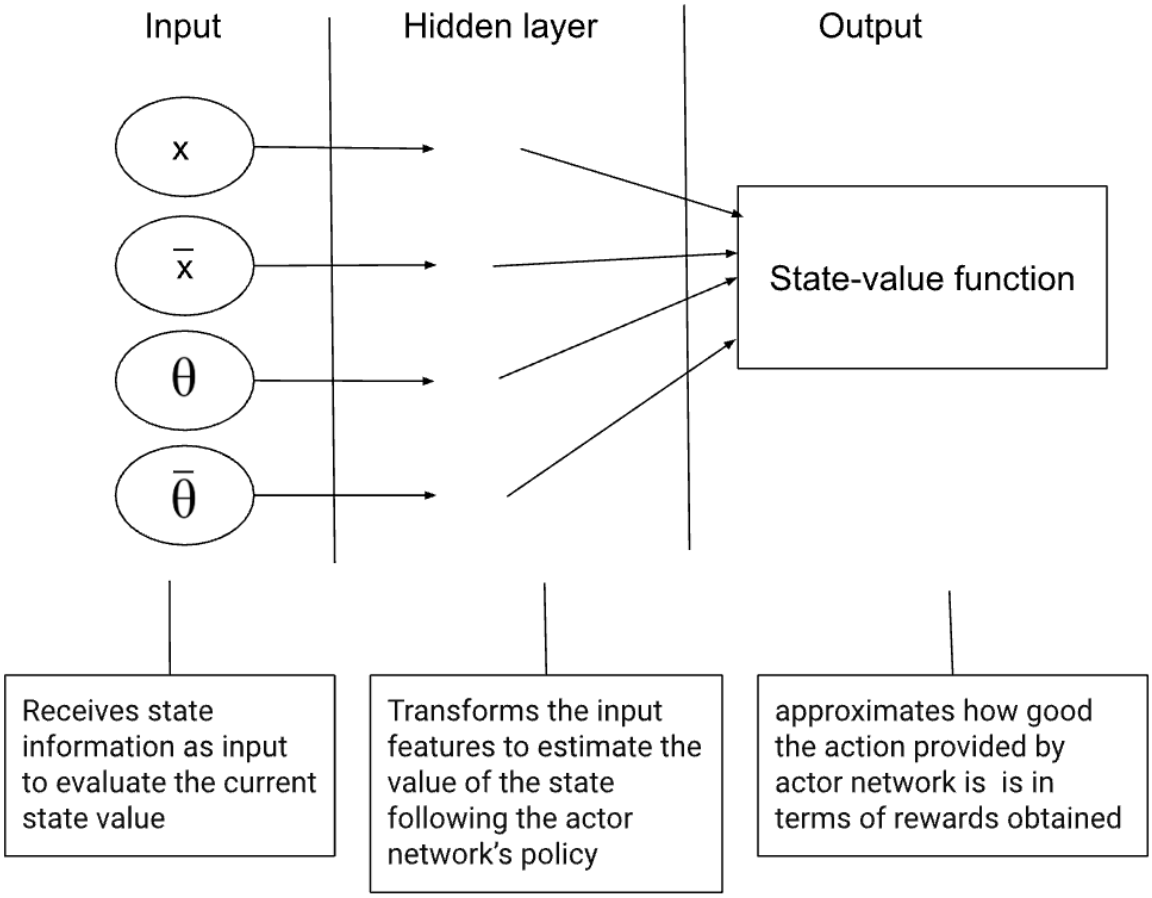

Figure 9. Critic neural network for PPO

PPO ensures that only small policy updates occur. To do this, a Kullback-Leibler (KL) Divergence[39] penalty, or a clipping function of the ratio of policy functions is used. However, the KL divergence penalty is often used in theoretical applications of PPO to measure how much a new policy diverges from an old policy. Thus, common practice is

[38] Karunakaran, D. (2020, September 30). *The actor-Critic Reinforcement Learning algorithm*. Intro to Artificial Intelligence. https://medium.com/intro-to-artificial-intelligence/the-actor-critic-reinforcement-learning-algorithm-c8095a655c14

[39] Ghosh, D. (2018, August 7). KL Divergence for Machine Learning. The RL Probabilist.

to use the clipping function as an alternative to KL divergence, for simpler implementation, which is what I have followed[40].

To update the PPO policy, the following equation in Figure 10 is used, to find the objective function, $L^{CLIP}(\theta)$ which is used to update the policy to improve the agent's performance. This objective function is how PPO balances exploration and exploitation, as the agent can try new actions, but cannot deviate too much from the existing policy.

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t\left[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t)\right]$$

Figure 10. PPO equation for updating policy[41].

Variable	Description
Advantage function, A_t[42]	Quantifies how much better (A_t is positive) or worse (A_t is negative) taking a action in a state is. The probability of taking the action increases when $A_t > 0$ and decreases when $A_t < 0$.
Ratio function, $r_t(\theta)$	$r_t(\theta)$ is the ratio between the policy function of the old policy and new policy. $\mathrm{r}(\theta)$ is calculated as such[43], $$r(\theta) = \frac{\pi_{new}(a\|s)}{\pi_{old}(a\|s)}$$ If r(θ) > 1, the action is more likely in the new policy than old policy. If r(θ) < 1, the action is less likely in the new policy than the old policy.
$\pi(a\|s)$	$\pi(a\|s)$ is the probability of taking an action, *a,* in a state, *s.*
[1−ϵ,1+ϵ]	Range in which r(θ) is clipped, to limit large changes in policy updates. ϵ is 0.2 in general practice. Since A_t achieved by updates resulting in r(θ) outside the clipping range is not used to update the policy, there is an incentive to not stray from the old policy[44].

Table 6. PPO equation variables[45].

[40] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Openai, O. (2017). Proximal Policy Optimization Algorithms. https://arxiv.org/pdf/1707.06347.pdf

[41] Simonini, T. (2022, August 5). Proximal Policy Optimization (PPO). Huggingface.co

[42] Schulman, J., Moritz, P., Levine, S., Jordan, M., & Abbeel, P. (2018). HIGH-DIMENSIONAL CONTINUOUS CONTROL USING GENERALIZED ADVANTAGE ESTIMATION. https://arxiv.org/pdf/1506.02438.pdf

[43] Simonini, T. (2022, August 5). Proximal Policy Optimization (PPO). Huggingface.co.

[44] Simonini, T. (2022, August 5). Proximal Policy Optimization (PPO). Huggingface.co.

[45] Simonini, T. (2022, August 5). Proximal Policy Optimization (PPO). Huggingface.co

In PPO the policy is updated as seen in Figure 11.

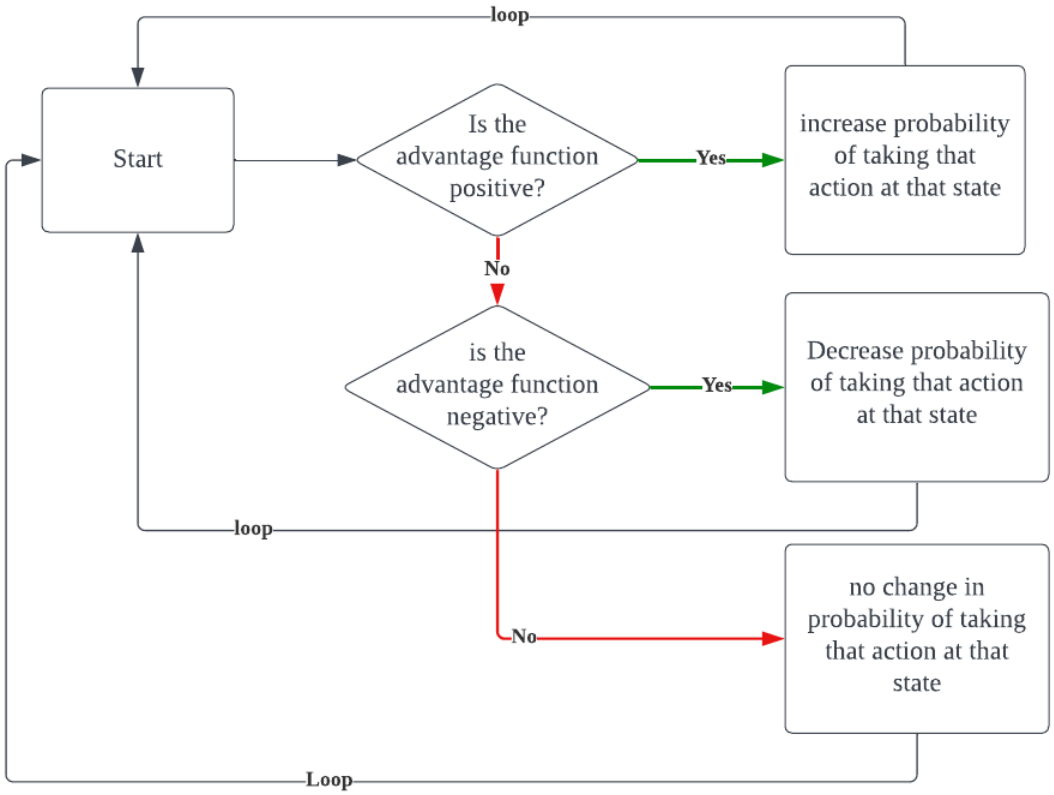

Figure 11. PPO policy updating process.

By repeating this process over multiple training iterations, PPO gradually improves the agent's policy, allowing it to make better decisions and maximize its rewards in the given environment.

// loop for all episodes

for episode in range(num_episodes):

Initialise cartVelocity, poleVelocity, cartPosition, poleAngle randomly

// Actor neural network outputs the probability distribution

Actor network outputs $\pi(a|s)$

// Critic network is used to compute the advantage

Compute advantage A_t

If A_t > 0

Probability of taking action increased in new policy

If A_t < 0

Probability of taking action decreased in new policy

End if

// Calculate ratio of new and old policy $r(\theta)$

$$r(\theta) = \frac{\pi_{new}(a|s)}{\pi_{old}(a|s)}$$

Clip $r(\theta)$ in range [1−ϵ,1+ϵ]

Update policy function using clipped $r(\theta)$ and A_t

End loop

Table 7. PPO pseudocode

Video 2. Self-generated experimental Proximal Policy Optimization video simulation for 200 Episodes
https://drive.google.com/file/d/1ReeeotGSB5mq7PTYMh3FUStJzBHxlii5/view?usp=sharing

Methodology

The implementation of both Q-learning and PPO algorithms were done using Python in Visual Studio (VS) Code. As mentioned earlier, in the cartpole environment, the states are continuous and defined as seen in Figure 12 according to OpenAI Gym documentation:

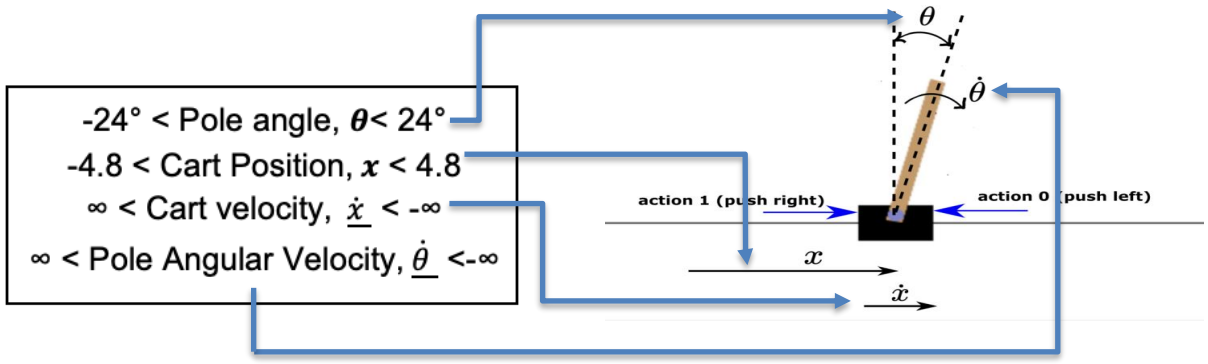

Figure 12. CartPole Variables[46]

The terminating conditions for each episode, consistent with documentation provided by OpenAI Gym, is seen in Table 8 below[47].

Terminating condition	Rationale
Pole angle is greater than 24°.	A pole angle greater than 24° likely leads to the pole toppling. Capping maximum deviation at 24° ensures agent maintains a stable position.
The position of the cart is greater than 4.8.	Ensures the agent does not make extreme movements to the left or the right to balance the pole, encouraging stable adjustment.
The maximum number of time steps for an episode before termination is 1000	Ensures a finite duration for each episode, to prevent the algorithm from running indefinitely. Finite time frame also encourages the agent to find optimal solutions efficiently.

Table 8. Terminating conditions[48]

[46] Haber, A. (2023b, January 31). *Detailed Explanation and Python Implementation of the Q-Learning Algorithm with Tests in Cart Pole OpenAI Gym Environment – Reinforcement Learning Tutorial – Fusion of Engineering, Control, Coding, Machine Learning, and Science*

[47] Cart Pole - Gym Documentation. (n.d.). Www.gymlibrary.dev.

[48] Cart Pole - Gym Documentation. (n.d.). Www.gymlibrary.dev.

Q-learning

However, since my chosen cartpole environment is a continuous environment, and Q-learning can handle only discrete action spaces, I limited cart velocity and pole angular velocity as seen in Table 9. These values were chosen as extremely high values of cart and pole angular velocity are unrealistic in practical applications.

-3 < Cart velocity < 3
-10 < Pole Angular Velocity < 10

Table 9. Limiting values into ranges

Figure 13 shows the code implementation of how the *ranges were limited.*

```
# discretising by limiting cart velocity and pole velocity. Cart position and pole angle is
# already limited according to OpenAI Gym documentation
cartVeloctyMax=3
cartVeloctyMin=-3
poleVelocityMin=-10
poleVelocityMax=10
upperBounds[1]=cartVeloctyMax
upperBounds[3]=poleVelocityMax
lowerBounds[1]=cartVeloctyMin
lowerBounds[3]=poleVelocityMin
```

Figure 13. Limiting ranges of values[49]

[49] *Refer to appendix for rest of code*

To discretise these ranges, I then divided these intervals into 20 segments each, such that it would be compatible with the Q-learning algorithm. The Figure 14 below shows how discretisation is used in Q-learning:

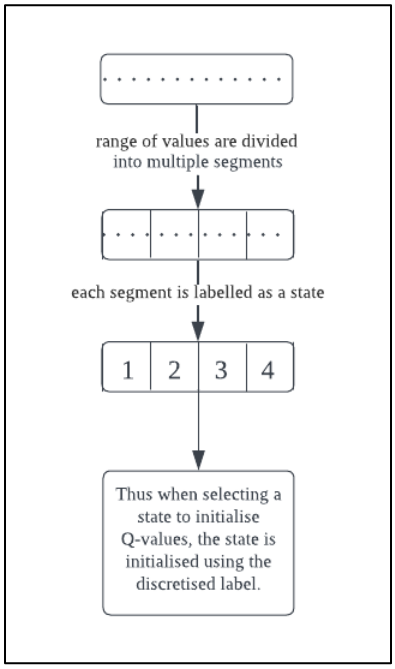

Figure 14. Discretisation

For example, if cart velocity is in the range (-3, 3), it would be in discretized as seen in Table 10 below:

Interval 1 = (-3, -2.7]
Interval 2 = (-2.7, -2.4]
Interval 3 = (-2.4, -2.1]
.
.
.
Interval 20 = (2.6, 3)

Table 10. Discretisation of variables in CartPole

If cart velocity was -2.5, it would be in interval 2. I divided the intervals into 20 segments as it provides a moderate level of granularity. When adjusting the number of intervals

through trial and error, I found 20 to be best for ensuring the algorithm would not take too long to run, while also being able to increase rewards based on action-state values. It's not too wide, allowing the algorithm to distinguish between different states, but not overly fine-grained, which necessitates intensive computational resources. Additionally, when researching the Q-learning algorithm in cartpole, the referenced codes used 20 intervals as well[50]. The various variables used, and their values are seen in Table 11.

Variable	Symbol	Value
Learning rate	α	0.1
Discount rate	*γ*	1
Epsilon parameter	*e*	0.2
Action-state value	*Q(s,a)*	-

Table 11. Q learning parameters

PPO

Unlike Q-learning, PPO can handle continuous action spaces like that in cartpole, so I did not need to discretise states. The various variables used, and their values are seen in Table 12.

Variable	Symbol	Value
Clipping value	ϵ	0.2
Advantage function	A_t	-
Ratio function	$r_t(\theta$	-
Policy function	$\pi_{new}(a\|s)$	-

Table 12. PPO parameters

[50] CartPole-v0/cartPole.py at master · JackFurby/CartPole-v0. (n.d.). GitHub. Retrieved February 10, 2024, from https://github.com/JackFurby/CartPole-v0/blob/master/cartPole.py

Experimental Procedure

Set the maximum reward as 1000, for both algorithms. Since the values of the obtained reward for each episode range from 10^1 to 10^3, I used a logarithmic scale for plotting the graphs of reward against episode. The deliberate selection of 200, 1000, and 3000 episodes in the experimental trials was based on preliminary trials. Trials revealed substantial differences in reward obtained between simulations comprising 200 and 1000 episodes, suggesting a point of analysis. Additionally, when extending episodes beyond 3000, there were fewer variations in reward obtained, and the time taken for the experimental trials became extremely long.

Steps:

1. Experimental training runs of 200 episodes were carried out for each algorithm.
2. The environment for each algorithm was reset to initial random values in the range (-0.1, 0.1) after each episode was completed.
3. The total reward obtained by each agent in each episode was collected.
4. The execution time of each algorithm for executing 200 episodes was collected.
5. The above steps were repeated for 1000 and 3000 training episodes respectively.

Raw data

Episode vs Reward

When collecting data of the number of episodes against reward for both algorithms, over 8000 data points were collected during the investigation. Thus, an extract of the raw data collected in the appendix is seen in Table 13.

Episode 1 || Avg rewards 21.0
Episode 2 || Avg rewards 15.0
Episode 3 || Avg rewards 13.0
Episode 4 || Avg rewards 27.0
Episode 5 || Avg rewards 15.0
Episode 6 || Avg rewards 64.0
.
.
.

Table 13. Raw data of episode and reward obtained.

Execution time

When finding the execution time of the algorithms, the elapsed time is collected by collecting the start time at the beginning of execution and the end time at the end of execution. The elapsed time is then derived using the Python code in Figure 15.

```
68      #end time of execution
69      et = time.time()
70      # time taken is end time minus the start time
71      elasped_time = et - st
72      print("Execution time:", elasped_time)
```

Figure 15. Execution time code[51]

The resulting data for each set of episodes was collected and is seen in Table 14 and Table 15 for Q Learning and PPO respectively.

number of episodes	execution time (s)
200	8.843794107
1000	32.07821107
3000	83.19758701

Table 14. Number of episodes and execution time of Q-learning

number of episodes	execution time (s)
200	21.93049502
1000	128.972877
3000	311.980607

Table 15. Number of episodes and execution time of PPO

[51] *Refer to appendix for rest of code.*

Memory Utilization

To tabulate the memory usage when executing PPO and Q -learning for 1000 episodes each, the memory usage was found using the memory profiler[52], to find the amount of memory each programme takes up during execution, as seen in Figure 16.

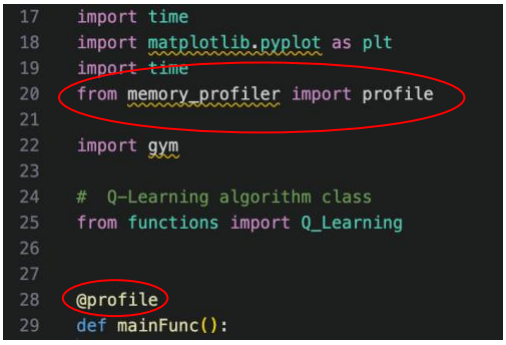

Figure 16. Function to find memory usage of algorithms[53].

The data collected regarding memory usage of the two algorithms is seen in Table 16.

	Memory Usage during execution of algorithms for 1000 episodes (MiB)
PPO	309.2
Q Learning	88.0

Table 16. Memory usage of PPO and Q-learning in execution of 1000 episodes

CPU Utilisation

When calculating the CPU utilization, I ran a separate program as seen in Figure 17[54], in a separate window which detects and outputs the CPU usage. When concurrently running

[52] memory_profiler: How to Profile Memory Usage in Python? by Sunny Solanki. (n.d.). Coderzcolumn.com. Retrieved September 30, 2023, from https://coderzcolumn.com/tutorials/python/how-to-profile-memory-usage-in-python-using-memory-profiler

[53] *Refer to appendix for rest of code*

[54] How to get current CPU and RAM usage in Python? (2021, January 22). GeeksforGeeks. https://www.geeksforgeeks.org/how-to-get-current-cpu-and-ram-usage-in-python/

the algorithms, 1000 episodes each, the CPU usage was found for each time interval of 1 second.

```python
import psutil
import time

while True:
    cpu_percent = psutil.cpu_percent(interval=1)
    print(f"CPU Uage: {cpu_percent}%")
    time.sleep(0)
```

Figure 17. Code for finding CPU usage of algorithms during execution[55]

The raw data for CPU utilization for Q-learning and PPO for the first 20 seconds of execution are seen in Table 17 and 18 respectively.

Execution Time (s)	CPU Usage(%)	Execution time(s)	CPU Usage (%)
1	10	11	31.6
2	17.9	12	35.7
3	17.3	13	33.6
4	24.4	14	32.3
5	45.7	15	37.3
6	40.5	16	38.1
7	43.8	17	44.4
8	40.5	18	42.9
9	39.4	19	41.3
10	40.2	20	40.5

Table 17. CPU Usage for Q learning

[55] *Refer to appendix for rest of code.*

Execution Time (s)	CPU Usage(%)	Execution time(s)	CPU Usage (%)
1	12.9	11	98.8
2	13.2	12	97.9
3	20.6	13	92.9
4	43.3	14	98.9
5	17.8	15	99.3
6	89.7	16	90.3
7	98.9	17	95.1
8	99.4	18	82.7
9	98.9	19	76.8
10	99.8	20	74.7

Table 18. CPU Usage for PPO

Data Analysis

When analysing the data obtained, the performance of PPO and Q-learning was compared based on rewards across 200, 800 and 1000 episodes. Additionally, CPU usage, execution time, and memory consumption of the 2 algorithms were compared.

Analysis (Q learning)

200 Episodes Simulation

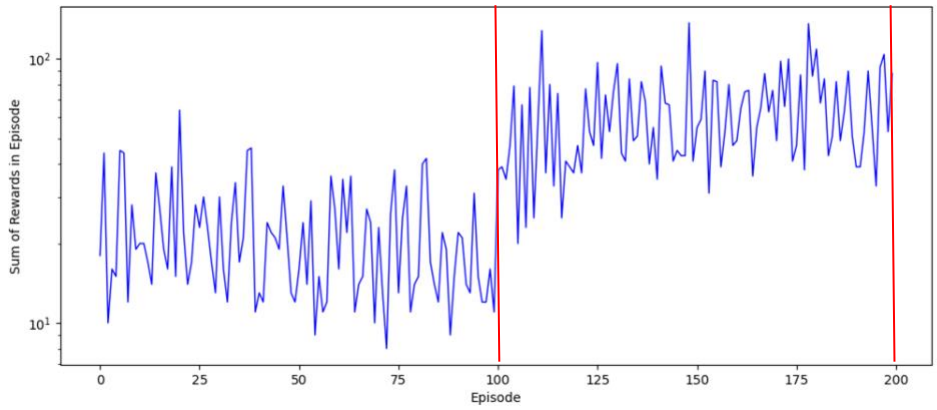

Figure 18. Rewards against episodes for 200 episodes

In Figure 18, from 0-100 episodes, rewards fluctuated throughout. For the first 100 episodes, the average reward obtained is roughly 80. There is a notable increase in rewards after 100 episodes. This suggests that the Q-learning algorithm has *started to converge towards a more effective policy as the agent has learned from its exploration.*

1000 Episodes Simulation

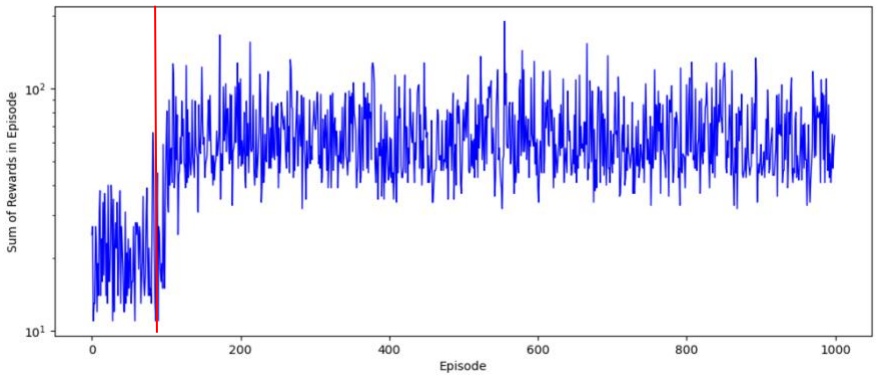

Figure 19. Rewards against episodes for 1000 episodes

In Figure 19, for the first 100 episodes, reward obtained is lower, due to exploration taking place. For subsequent episodes, epsilon-greedy approach takes place. Average reward obtained remains consistent at approximately 100, for the remaining episodes.

3000 Episodes Simulation

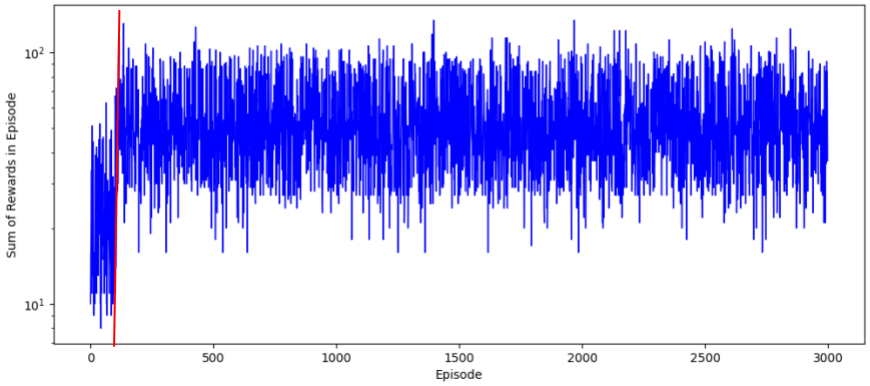

Figure 20. Rewards against episodes for 3000 episodes

In Figure 20, for the first 100 episodes, rewards exhibit substantial variability, *indicating that the agent is actively exploring the environment and learning different actions* and the resulting action-value functions. For subsequent episodes, rewards remain consistent at around 100. *Stability indicates the agent has found a policy that consistently achieves a satisfactory level of performance* in balancing the CartPole. Stagnation[56] of rewards could be *due to exploration decay*, which is the gradual reduction in the exploration rate over time[57]. If the agent starts with high exploration but decays too quickly, the *agent might have stopped exploring before exploring a more effective policy.*

[56] Tunstead, K. (2019). *Combating Stagnation in Reinforcement Learning Through "Guided Learning" With "Taught-Response Memory."* https://ceur-ws.org/Vol-2444/ialatecml_shortpaper1.pdf
[57] Nair, S. (2020, September 21). *RL Series#3: To explore or not to explore, that is the question*. Analytics Vidhya. https://medium.com/analytics-vidhya/rl-series-3-to-explore-or-not-to-explore-1ff88e4bf5af

Execution Time across episodes (Q-learning)

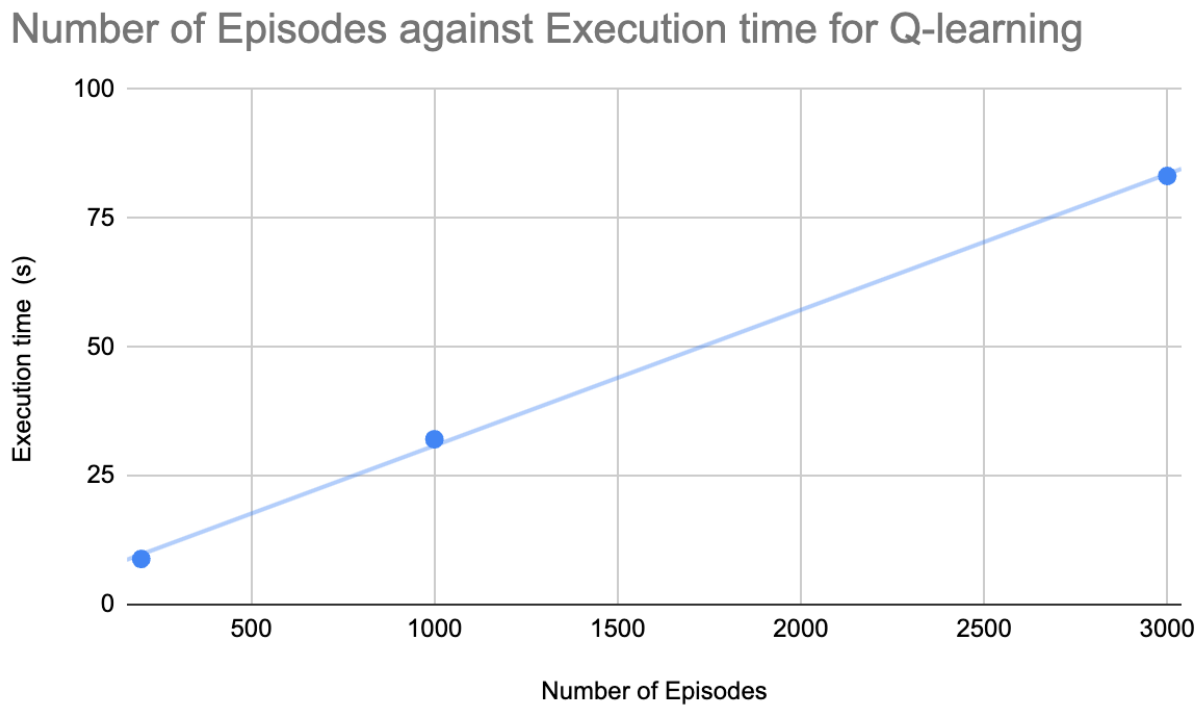

Figure 21. Execution time against number of episodes for Q-learning

Figure 21 shows a linear relationship between number of episodes and execution time for the algorithms.

Analysis (PPO)

200 Episodes Simulation

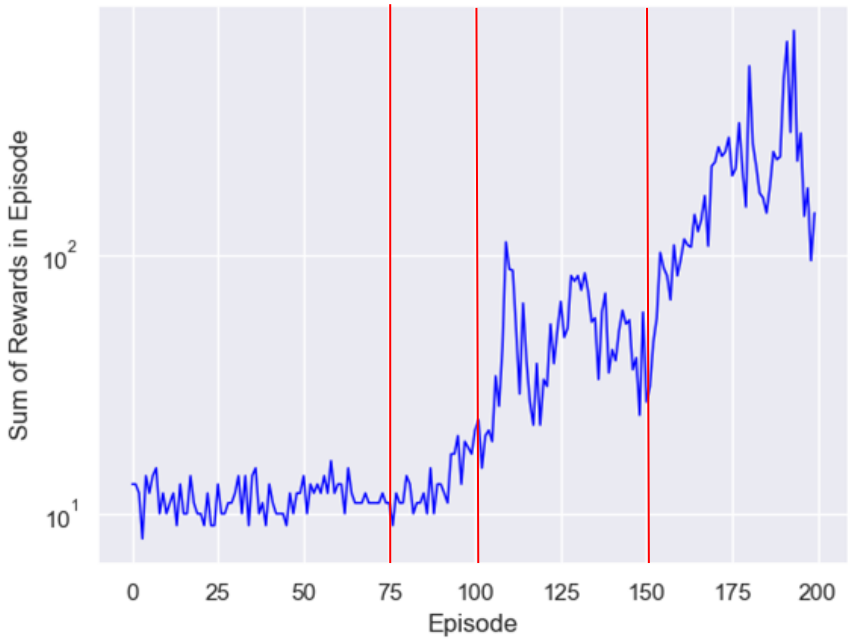

Figure 22. Episodes against reward for 200 episodes

In Figure 22, From 0 to 75 episodes, rewards remain low around 10. From 75 to 150 episodes, rewards increase more rapidly, just under 100. For subsequent episodes, rewards continue to increase rapidly and remains high, above 100 for the rest of episodes.

1000 Episodes Simulation

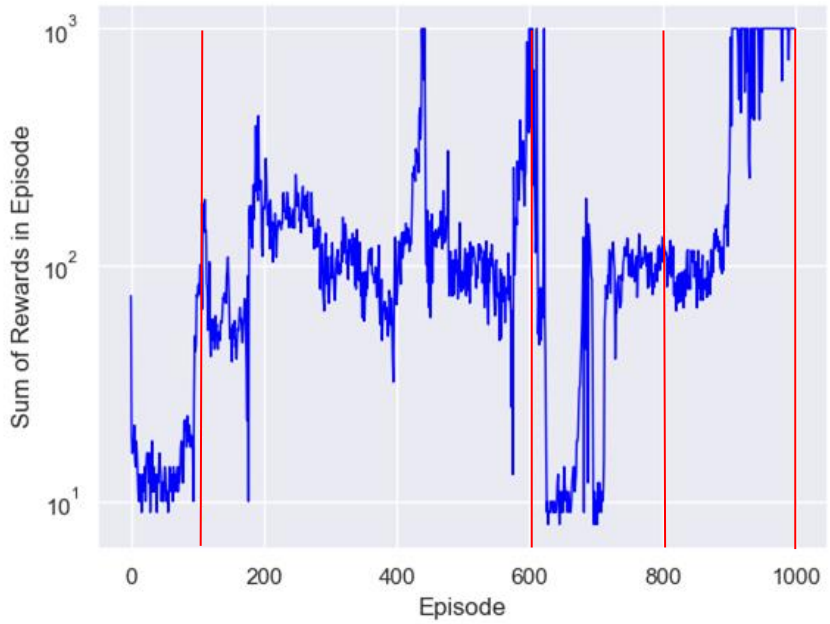

Figure 23. Episodes against reward for 1000 episodes

.

In Figure 23, from 0 to 100 episodes, rewards increase steadily from 0 to 100 episodes. From 100 to 600 episodes, rewards remain relatively consistent at an average of 100. From 600 to 800 episodes, there are 2 major dips in reward obtained, which could be *due to the agent taking consecutive actions that resulted in* a negative advantage. Subsequently, reward increases to the maximum possible of 1000. *This indicates that policy updates are converging to produce a more optimal policy.*

3000 Episodes Simulation

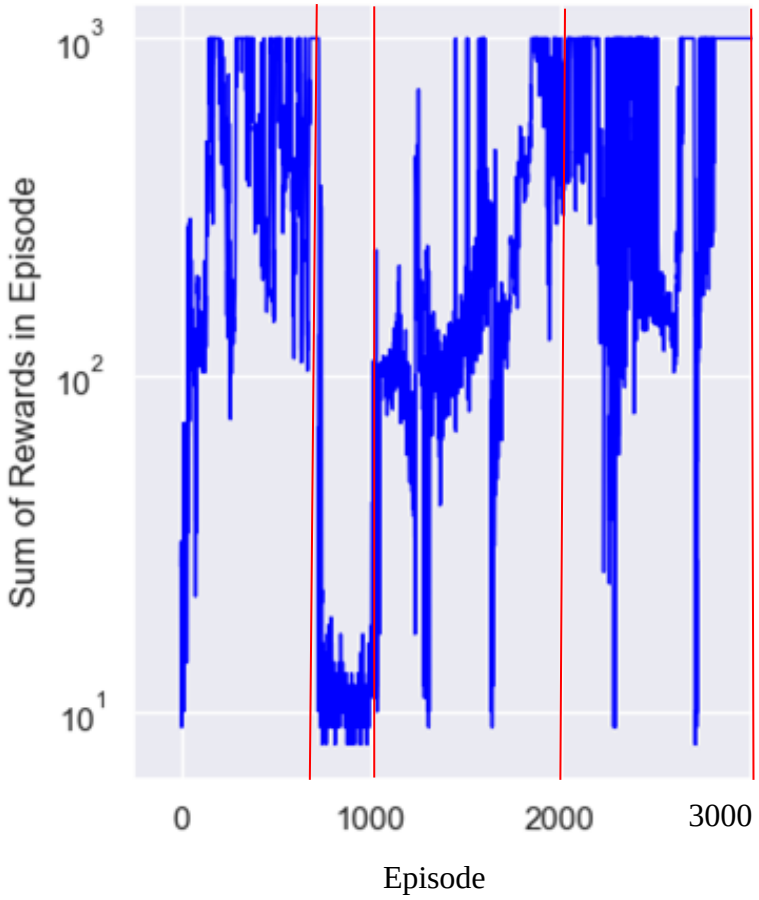

Figure 24. Episodes against reward for 3000 episodes

In Figure 24, from 0 to 800 episodes, there is a general increasing trend in reward as the number of episodes progresses. At the 800th episode, there is a drop in the reward. This could be due to the *agent taking consecutive actions resulting in a negative advantage.* From 800 to 3000 episodes, reward appears to stabilize and continue its upward trend, a sign that the *agent has continued obtaining positive advantages*. Maximum reward of 1000 is frequently obtained from 2000 to 3000. This suggests that *policy updates are converging*, for optimal policy.

Execution Time across episodes (PPO)

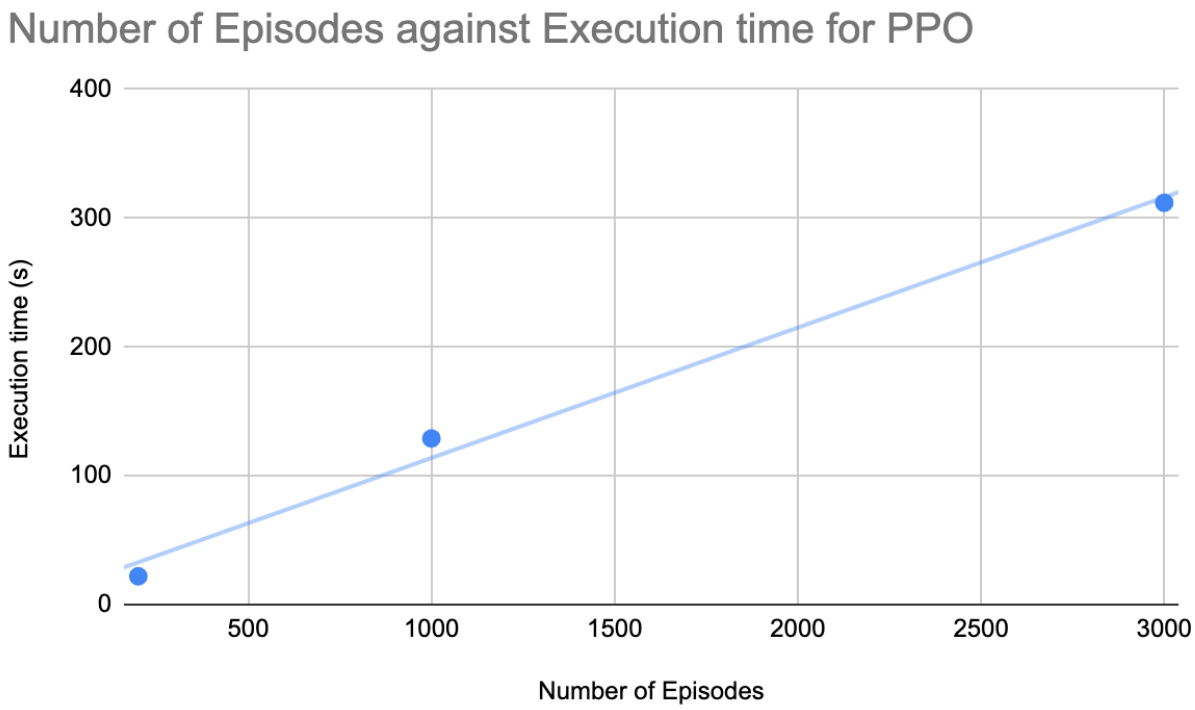

Figure 25. Number of episodes against execution time for PPO

Figure 25 shows a linear relationship between number of episodes and execution time for the algorithms.

Comparisons of PPO and Q-learning

200 Episodes Simulation

In the first 100 episodes, the random exploration of Q-learning yielded higher rewards, compared to the clipped policy function approach in PPO. Due to the clipped nature of the PPO algorithms, where each policy update is small, the *initial episodes yield lower rewards as the algorithm slowly updates the policy.* However, towards the end of the 200 episodes PPO yields higher rewards than Q-learning. The *epsilon greedy approach in Q-learning for the next episodes till 200 is not as effective as the clipped method in PPO.*

1000 Episodes Simulation

Q-learning consistently obtained rewards less than 100, while PPO obtained rewards more than 100 for majority of episodes.

However, Q-learning exhibits fewer major fluctuations, as its rewards remain stable after the initial 100 episodes without reverting to the levels observed during random exploration. From 600 to 800 episodes, there were 2 major dips in reward obtained for PPO. Q-learning's focus on estimating Q-values, representing expected rewards for action-state values, contributes to its stability. *Updates in Q-learning are driven by the epsilon greedy approach, resulting in smoother transitions*. Conversely, PPO directly optimizes policies, leading to *more variable policy updates due to their complex and nonlinear nature*. Moreover, PPO's use of *stochastic policies introduces variability, as agents randomly sample actions from probability distributions*, explaining the observed fluctuations compared to Q-learning.

3000 Episodes Simulation

In the span of 3000 episodes, both algorithms tend to stabilize, with Q learning stabilizing around the 200th episode, and PPO stabilizing around the 2000th episode. The earlier stabilization of Q-learning could be because *Q-learning has a simpler and more intuitive update mechanism*. It *converges more quickly to a stable action value*

function, and thus yields more stable rewards because it uses straightforward iterative updates. On the other hand, PPO updates from a stochastic policy. Thus, leading to *more variability in PPO compared to Q learning, and a later convergence*. PPO *performs better with more episodes as the numerous updates to the policy converge to the optimal policy*, leading to consistently high rewards.

Learning Rate

Across all episode trials, there is a consistent trend of PPO having a faster learning rate compared to Q-learning, yielding much higher rewards in a shorter period. This could be as *PPO optimizes the policy* to improve the decision-making of the agent. It updates the policy parameters based on the obtained advantage. This approach allows PPO *to focus on reinforcing actions leading to higher rewards, causing more rapid increases in obtained rewards.* Q-learning however, *updates the Q-values based on the epsilon-greedy approach which could require more iterations to converge to a higher reward.*

Execution time

Across the 3 different episode trials, PPO consistently shows a longer execution time than Q-learning, approximately 3 times slower. As mentioned previously, *Q-learning handles discrete action spaces, while PPO handles continuous action spaces*. The *continuous nature of PPO makes it more complex to train*, resulting in longer execution times. Moreover, PPO's *policy updates occur iteratively to optimize performance*, contrasting with *Q-learning's straightforward implementation of the epsilon greedy approach* Because epsilon-*greedy does not require the complex optimization* process of PPO, it is computationally faster.

Memory Usage:

When comparing the memory usage of the 2 algorithms, it is evident that PPO uses much more memory than Q-learning. Q-learning utilizes 88.0 MiB worth of memory, while PPO utilizes 309.2 MiB worth of memory. This is likely due to the structure of the algorithms. PPO typically involves representing the policy in a *neural network using the actor-critic*

model. In contrast, Q-learning usually involves a *Q-table, which is smaller than a neural network*.

CPU Utilisation:

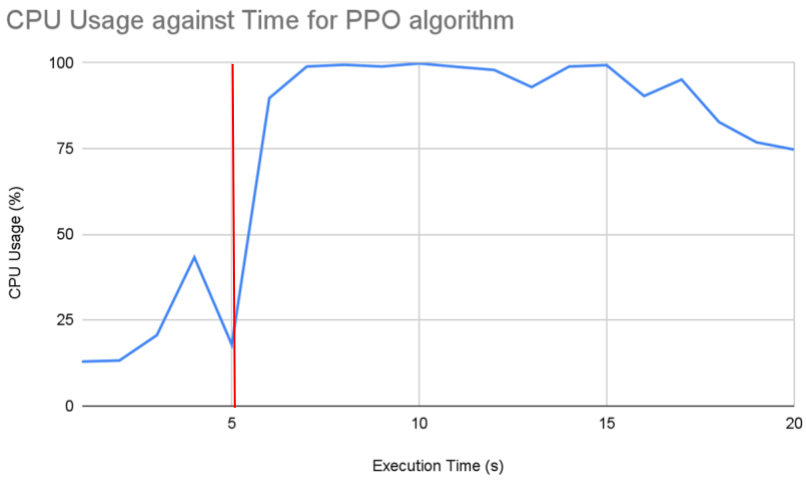

Figure 26. CPU usage of PPO

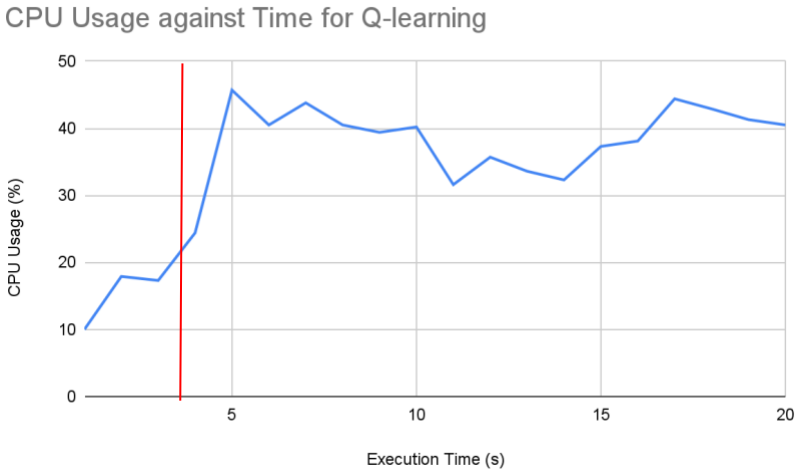

Figure 27. CPU usage of Q-learning

Since the run time of PPO was much longer than that of Q-learning, I took a 20 second period from the execution of each algorithm for 1000 episodes, to compare the CPU usage. At the 5 second mark of Figure 26 and the 3 second mark of Figure 27, the 2 reinforcement learning algorithms started the execution, resulting in the uptick in CPU

Usage. Evidently, there is a much higher CPU utilization for the execution of the PPO algorithm, compared to Q-learning. For PPO, the CPU usage was close to 100%, averaging around 90%. For the Q-learning algorithm the CPU usage was close to 50%, averaging around 40%. Thus, PPO uses more CPU processing power compared to Q-learning.

This is likely because of the *continuous nature of PPO making it more complex to train compared to Q-learning which handles discrete action spaces.* Additionally, PPO *updates its policy each time*, to optimize the policy. In contrast, Q-learning is simpler to implement as it *implements the epsilon greedy* approach. Because epsilon-greedy does not require the complex optimization process of policy-based methods like PPO, Q-learning utilizes less CPU processing power.

Conclusion

Overall, both Q-learning and PPO have their strengths and weaknesses, as summarized below in Table 19.

Factor	Q-learning	PPO
Learning Rate	Slower learning rate	Faster learning rate
Execution Time	Faster execution time than PPO.	Slower execution time than Q-learning
Memory Usage	Less memory used	More memory used
CPU Utilisation	Less CPU usage than PPO	More CPU memory usage than Q-learning

Table 19. Comparisons of experimental findings

PPO is more effective in balancing the pole on the cart for a longer period. PPO is faster in terms of learning rate than Q-learning and can learn the policy to balance the pole on the cart more quickly. Q-learning is faster to execute but performs worse in terms of reward obtained. However, in terms of CPU and memory utilization, as well as execution time, Q-learning performs considerably better than PPO. Working to optimize the PPO algorithm to execute faster and use less processing power and memory could lead to the optimal algorithm, combining the benefits of both PPO and Q-learning.

For environments with continuous action spaces, including CartPole, PPO is more effective than Q-learning, since the PPO algorithm yielded higher rewards for most episodes, and was a more effective policy. For example, for applications such as robotics, or control systems, where precise control over continuous actions is needed, PPO could perform better. Contrastingly, in applications such as simple games with discrete actions, or inventory management systems, Q-learning could be better as it is easier to set up and is more time and memory efficient.

Limitations

This investigation is limited due to the use of a predictable environment. The cartpole environment is very simple and predictable, as it consists only of 4 variables and only 2 actions. The performance of the two algorithms in such a predictable environment like the cartpole environment may not generalize well to more complex real-world scenarios. Additionally, I was limited by the processing power of my device. Due to limited processing power, there was a limited number of episodes that could be run. I hypothesize that Q-learning would eventually reach a higher number of rewards, given more episodes to run for, however, the capabilities of my device were limited in testing this.

Further Studies

To address the issue of rewards stagnating in Q-learning, a means of random exploration throughout the series of episodes could be implemented, especially when the rewards seem to be stagnating, to allow for more informed decisions regarding actions. This investigation can further be extended by testing PPO and Q-learning on other OpenAI environments to see how the different algorithms work in different environments.

Bibliography

1. *A Brief Introduction to Proximal Policy Optimization*. (2021, September 1). GeeksforGeeks. https://www.geeksforgeeks.org/a-brief-introduction-to-proximal-policy-optimization/#:~:text=The%20most%20common%20implementation%20of
2. baeldung. (2020, December 18). *Epsilon-Greedy Q-learning | Baeldung on Computer Science*. Www.baeldung.com. https://www.baeldung.com/cs/epsilon-greedy-q-learning
3. Bajaj, P. (2018, April 25). *Reinforcement learning - GeeksforGeeks*. GeeksforGeeks. https://www.geeksforgeeks.org/what-is-reinforcement-learning/
4. Brown, S. (2021, April 21). *Machine learning, explained*. MIT Sloan; MIT Sloan School of Management. https://mitsloan.mit.edu/ideas-made-to-matter/machine-learning-explained
5. Brownlee, J. (2019, June 4). *How to Configure the Learning Rate Hyperparameter When Training Deep Learning Neural Networks*. Machine Learning Mastery. https://machinelearningmastery.com/learning-rate-for-deep-learning-neural-networks/
6. Candidate, C. M. B., Ph D. (2022, September 20). *Strategies for Decaying Epsilon in Epsilon-Greedy*. Medium. https://medium.com/@CalebMBowyer/strategies-for-decaying-epsilon-in-epsilon-greedy-9b500ad9171d
7. *Cart Pole - Gym Documentation*. (n.d.). Www.gymlibrary.dev. https://www.gymlibrary.dev/environments/classic_control/cart_pole/
8. *CartPole-v0/cartPole.py at master · JackFurby/CartPole-v0*. (n.d.). GitHub. Retrieved February 10, 2024, from https://github.com/JackFurby/CartPole-v0/blob/master/cartPole.py

9. DhanushKumar. (2024, February 21). *PPO Algorithm - DhanushKumar - Medium*. Medium; Medium. https://medium.com/@danushidk507/ppo-algorithm-3b33195de14a
10. DHMINE, O. (2023, November 23). *Q-Learning and PPO: Driving Forces in OpenAI's AI Mastery*. Medium. https://medium.com/@oumoudhmine/q-learning-and-ppo-driving-forces-in-openais-ai-mastery-655027c8670f
11. Doshi, K. (2021, February 14). *Reinforcement Learning Explained Visually (Part 4): Q Learning, step-by-step*. Medium. https://towardsdatascience.com/reinforcement-learning-explained-visually-part-4-q-learning-step-by-step-b65efb731d3e
12. Galarnyk, M., & Mika, S. (2021, August 6). *Anyscale - An Introduction to Reinforcement Learning with OpenAI Gym, RLlib, and Google Colab*. Anyscale. https://www.anyscale.com/blog/an-introduction-to-reinforcement-learning-with-openai-gym-rllib-and-google
13. Ghosh, D. (2018, August 7). *KL Divergence for Machine Learning*. The RL Probabilist. https://dibyaghosh.com/blog/probability/kldivergence.html
14. *Google Colab*. (2024). Google.com. https://colab.research.google.com/drive/1MsRlEWRAk712AQPmoM9X9E6bNeHULRDb
15. Haber, A. (2023a, January 21). *Explanation and Python Implementation of On-Policy SARSA Temporal Difference Learning – Reinforcement Learning Tutorial with OpenAI Gym – Fusion of Engineering, Control, Coding, Machine Learning, and Science*. https://aleksandarhaber.com/explanation-and-python-implementation-of-on-policy-sarsa-temporal-difference-learning-reinforcement-learning-tutorial/

16. Haber, A. (2023b, January 31). *Detailed Explanation and Python Implementation of the Q-Learning Algorithm with Tests in Cart Pole OpenAI Gym Environment – Reinforcement Learning Tutorial – Fusion of Engineering, Control, Coding, Machine Learning, and Science*. https://aleksandarhaber.com/q-learning-in-python-with-tests-in-cart-pole-openai-gym-environment-reinforcement-learning-tutorial/
17. *How to get current CPU and RAM usage in Python?* (2021, January 22). GeeksforGeeks. https://www.geeksforgeeks.org/how-to-get-current-cpu-and-ram-usage-in-python/
18. *Introduction to RL and Deep Q Networks | TensorFlow Agents*. (2023). TensorFlow. https://www.tensorflow.org/agents/tutorials/0_intro_rl#:~:text=The%20Cartpole%20environment%20is%20one
19. Karunakaran, D. (2020, September 30). *The actor-Critic Reinforcement Learning algorithm*. Intro to Artificial Intelligence. https://medium.com/intro-to-artificial-intelligence/the-actor-critic-reinforcement-learning-algorithm-c8095a655c14
20. Kumar, S. (2020). *Balancing a CartPole System with Reinforcement Learning -A Tutorial*. https://arxiv.org/pdf/2006.04938.pdf
21. *memory_profiler: How to Profile Memory Usage in Python? by Sunny Solanki*. (n.d.). Coderzcolumn.com. Retrieved September 30, 2023, from https://coderzcolumn.com/tutorials/python/how-to-profile-memory-usage-in-python-using-memory-profiler
22. Nair, S. (2020, September 21). *RL Series#3: To explore or not to explore, that is the question*. Analytics Vidhya. https://medium.com/analytics-vidhya/rl-series-3-to-explore-or-not-to-explore-1ff88e4bf5af

23. Or, B. (2021, January 31). *Value-based Methods in Deep Reinforcement Learning*. Medium. https://towardsdatascience.com/value-based-methods-in-deep-reinforcement-learning-d40ca1086e1
24. PhD, W. van H. (2023, January 31). *Proximal Policy Optimization (PPO) Explained*. Medium. https://towardsdatascience.com/proximal-policy-optimization-ppo-explained-abed1952457b
25. *Proximal Policy Optimization — Spinning Up documentation*. (n.d.). Spinningup.openai.com. Retrieved September 30, 2023, from https://spinningup.openai.com/en/latest/algorithms/ppo.html#id5
26. Santhosh, S. (2023, January 2). *Reinforcement Learning (Part-8): Proximal Policy Optimization(PPO) for trading…*. Medium. https://medium.com/@sthanikamsanthosh1994/reinforcement-learning-part-8-proximal-policy-optimization-ppo-for-trading-9f1c3431f27d#:~:text=PPO%20is%20a%20type%20of
27. Schulman, J., Moritz, P., Levine, S., Jordan, M., & Abbeel, P. (2018). *HIGH-DIMENSIONAL CONTINUOUS CONTROL USING GENERALIZED ADVANTAGE ESTIMATION*. https://arxiv.org/pdf/1506.02438.pdf
28. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Openai, O. (2017). *Proximal Policy Optimization Algorithms*. https://arxiv.org/pdf/1707.06347.pdf
29. Shyalika, C. (2021, July 13). *A Beginners Guide to Q-Learning*. Medium. https://towardsdatascience.com/a-beginners-guide-to-q-learning-c3e2a30a653c#:~:text=Q%2Dlearning%20is%20a%20values

30. Simonini, T. (2022, August 5). *Proximal Policy Optimization (PPO)*. Huggingface.co. https://huggingface.co/blog/deep-rl-ppo

31. Sutton, R., & Barto, A. (n.d.). *Reinforcement Learning An Introduction second edition*. https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf

32. Taulli, T. (n.d.). *Reinforcement Learning: The Next Big Thing For AI (Artificial Intelligence)?* Forbes. Retrieved November 17, 2023, from https://www.forbes.com/sites/tomtaulli/2020/06/05/reinforcement-learning-the-next-big-thing-for-ai-artificial-intelligence/?sh=28465b9f62ba

33. *The Exploration/Exploitation trade-off - Hugging Face Deep RL Course*. (n.d.). Huggingface.co. Retrieved September 30, 2023, from https://huggingface.co/learn/deep-rl-course/unit1/exp-exp-tradeoff

34. Tunstead, K. (n.d.). *Combating Stagnation in Reinforcement Learning Through "Guided Learning" With "Taught-Response Memory."* Retrieved May 10, 2024, from https://ceur-ws.org/Vol-2444/ialatecml_shortpaper1.pdf

35. Tunstead, K. (2019). *Combating Stagnation in Reinforcement Learning Through "Guided Learning" With "Taught-Response Memory."* https://ceur-ws.org/Vol-2444/ialatecml_shortpaper1.pdf

36. ujjwalkarn. (2016, August 10). *A Quick Introduction to Neural Networks*. The Data Science Blog; the data science blog. https://ujjwalkarn.me/2016/08/09/quick-intro-neural-networks/

37. Xu, H., Yan, Z., Xuan, J., Zhang, G., & Lu, J. (2023). Improving Proximal Policy Optimization with Alpha Divergence. *Neurocomputing*. https://doi.org/10.1016/j.neucom.2023.02.008.

Appendix

(Note: All code was implemented in Visual Studio Code, using Python)

Q Learning Functions[58]:

```
import numpy as np

class Q_Learning:

   def __init__(self, env, alpha, gamma, epsilon, numberEpisodes, Intervals,
lowerBounds, upperBounds):
      import numpy as np

      self.env = env

      # the learning rate
      self.alpha=alpha

      # the discount rate
      self.gamma = gamma

      # the epsilon value for implementing epsilon greedy
      self.epsilon = epsilon

      self.action = env.action_space.n

      # the number of episodes
      self.numberEpisodes = numberEpisodes

      self.Intervals = Intervals
      self.lowerBounds = lowerBounds
      self.upperBounds = upperBounds

      #sum of rewards in every learning episode
      self.sumRewardsEpisode = []

      # action value function matrix
```

[58] Adapted from:
Haber, A. (2023b, January 31). *Detailed Explanation and Python Implementation of the Q-Learning Algorithm with Tests in Cart Pole OpenAI Gym Environment – Reinforcement Learning Tutorial – Fusion of Engineering, Control, Coding, Machine Learning, and Science*. https://aleksandarhaber.com/q-learning-in-python-with-tests-in-cart-pole-openai-gym-environment-reinforcement-learning-tutorial/

```
        self.Qmatrix = np.random.uniform(low = 0, high =1,
size=(Intervals[0],Intervals[1],Intervals[2],Intervals[3],self.action))


    def returnIndexState(self,state):

        #parameters

cartPositionInt=np.linspace(self.lowerBounds[0],self.upperBounds[0],self.Interv
als[0])

cartVelocityInt=np.linspace(self.lowerBounds[1],self.upperBounds[1],self.Interv
als[1])

poleAngleInt=np.linspace(self.lowerBounds[2],self.upperBounds[2],self.Interval
s[2])

poleAngleVelocityInt=np.linspace(self.lowerBounds[3],self.upperBounds[3],self.
Intervals[3])

        indexOfPosition=np.maximum(np.digitize(state[0],cartPositionInt)-1,0)
        indexOfVelocity=np.maximum(np.digitize(state[1],cartVelocityInt)-1,0)
        indexOfAngle=np.maximum(np.digitize(state[2],poleAngleInt)-1,0)

indexOfAngularVelocity=np.maximum(np.digitize(state[3],poleAngleVelocityInt)-
1,0)

        return
tuple([indexOfPosition,indexOfVelocity,indexOfAngle,indexOfAngularVelocity])

    def selectAction(self,state,episodeNum):

        # first 100 episodes are selected randomly for exploration
        if episodeNum<100:
            return np.random.choice(self.action)

        # this number is used for the epsilon greedy approach, to compare to
epsilon parameter
        # randomNumber is selected in range of 0 to 1

        randomNumber=np.random.random()
```

```
        # after 100 episodes, the epsilon parameter is decreased for epsilon-
greedy to take place
        if episodeNum>100:
            self.epsilon=0.999*self.epsilon

        # if random number is more than the epsilon parameter, random choice of
action
        if randomNumber < self.epsilon:
            # returns a random action
            return np.random.choice(self.action)

        # if random number is less than the epsilon parameter, greedy choice of
action
        else:
            # select the action that gives the maximum Q-value
            return
np.random.choice(np.where(self.Qmatrix[self.returnIndexState(state)]==np.max
(self.Qmatrix[self.returnIndexState(state)]))[0])


    def simulateEpisodes(self):
        import numpy as np
        # loop till final episode
        for indexEpisode in range(self.numberEpisodes):
            rewardsEpisode=[]

            # reset the environment after each episode terminates
            (stateS,_)=self.env.reset()
            stateS=list(stateS)

            terminalState=False
            while not terminalState:

                stateSIndex=self.returnIndexState(stateS)

                # select an action on the basis of the current state, denoted by stateS
                actionA = self.selectAction(stateS,indexEpisode)

                (stateSprime, reward, terminalState,_,_) = self.env.step(actionA)

                rewardsEpisode.append(reward)
```

```
            stateSprime=list(stateSprime)

            stateSprimeIndex=self.returnIndexState(stateSprime)

            QmaxPrime=np.max(self.Qmatrix[stateSprimeIndex])

self.Qmatrix[stateSIndex+(actionA,)]=self.Qmatrix[stateSIndex+(actionA,)]+self.
alpha*(reward+self.gamma*QmaxPrime-self.Qmatrix[stateSIndex+(actionA,)])
            else:
                # in the terminal state Q-value is zero
                 Qmatrix[stateSprime,actionAprime]=0

self.Qmatrix[stateSIndex+(actionA,)]=self.Qmatrix[stateSIndex+(actionA,)]+self.
alpha*(reward-self.Qmatrix[stateSIndex+(actionA,)])

            # change current state to the next state (denoted as Sprime)
            stateS=stateSprime

        print("Episode {} |".format(indexEpisode) + " Avg Rewards {} "
.format(np.sum(rewardsEpisode)))
        self.sumRewardsEpisode.append(np.sum(rewardsEpisode))
```

Q Learning Main Python Code[59]:

```
import sys
sys.path.append('/Library/Frameworks/Python.framework/Versions/3.9/lib/python3.9/site-packages')
import numpy as np
import time
import matplotlib.pyplot as plt
import time
from memory_profiler import profile

import gym

from functions import Q_Learning

#memory profiler to find amount of memory used by the algorithm during execution
@profile

def mainFunc():

    # start time of execution
    st = time.time()

    env=gym.make('CartPole-v1')
    (state,_)=env.reset()

    # parameters of cart position, cart velocity, pole angle, and pole angular velocity
    upperBounds=env.observation_space.high
    lowerBounds=env.observation_space.low

    # discretising by limiting cart velocity and pole velocity. Cart position and pole angle is
```

[59] Adapted from:
Haber, A. (2023b, January 31). *Detailed Explanation and Python Implementation of the Q-Learning Algorithm with Tests in Cart Pole OpenAI Gym Environment – Reinforcement Learning Tutorial – Fusion of Engineering, Control, Coding, Machine Learning, and Science*. https://aleksandarhaber.com/q-learning-in-python-with-tests-in-cart-pole-openai-gym-environment-reinforcement-learning-tutorial/

```
    # already limited according to OpenAI Gym documentation for cartpole
    cartVeloctyMax=3
    cartVeloctyMin=-3
    poleVelocityMin=-10
    poleVelocityMax=10
    upperBounds[1] = cartVelocityMax
    lowerBounds[1] = cartVeloctyMin
    upperBounds[3] = poleVelocityMax
    lowerBounds[3] = poleVelocityMin

    #dicretisation of pole angle, cart velocity, cart position, and pole angular
velocity for Q-learning, which only handles discrete
    # separate them into 20 intervals, within the ranges
    intCPosition=20
    intCVelocity=20
    intPAngle=20
    intPVelocity=20
    Intervals=[intCartPosition,intCartVelocity,intPoleAngle,intPoleVelocity]

    # define parameters according to (Kumar, 2020)[60]

# learning rate
    alpha=0.1

# discount rate
    gamma=1

# epsilon for epsilon-greedy
    epsilon=0.2
    numberEpisodes=200


Q1=Q_Learning(env,alpha,gamma,epsilon,numberEpisodes,Intervals,lowerBou
nds,upperBounds)
    # run the Q-Learning algorithm
    Q1.simulateEpisodes()

    #end time of execution
    et = time.time()
```

[60] Kumar, S. (2020). *Balancing a CartPole System with Reinforcement Learning -A Tutorial*. https://arxiv.org/pdf/2006.04938.pdf

```
    # time taken is end time minus the start time
    elasped_time = et - st
    print("Execution time:", elasped_time)

    plt.figure(figsize=(12, 5))

    # plot graph of episode against reward
    plt.plot(Q1.sumRewardsEpisode,color='blue', linewidth = 1)
    plt.xlabel('Episode')
    plt.ylabel('Sum of Rewards in Episode')
    plt.yscale('log')
    plt.savefig(‘graph.png')
    plt.show()

mainFunc()
```

PPO Python Code[61]:

```
import gym
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
import torch
from torch import nn
from torch import optim
from torch.distributions.categorical import Categorical
import time
from memory_profiler import profile

import sys
st = time.time()
sys.path.append('/Library/Frameworks/Python.framework/Versions/3.9/lib/python3.9/site-packages')
sns.set()

DEVICE = 'cpu'

# actor and critic neural network, defining the layers
class ActorCriticNetwork(nn.Module):
  def __init__(self, obs_space_size, action_space_size):
    super().__init__()

    self.shared_layers = nn.Sequential(
        nn.Linear(obs_space_size, 64),
        nn.ReLU(),
        nn.Linear(64, 64),
        nn.ReLU())

    self.policy_layers = nn.Sequential(
        nn.Linear(64, 64),
        nn.ReLU(),
        nn.Linear(64, action_space_size))

    self.value_layers = nn.Sequential(
        nn.Linear(64, 64),
```

[61] Adapted from:
Google Colab. (2024). Google.com. https://colab.research.google.com/drive/1MsRlEWRAk712AQPmoM9X9E6bNeHULRDb

```
        nn.ReLU(),
        nn.Linear(64, 1))

  def value(self, obs):
    layers = self.shared_layers(obs)
    value = self.value_layers(layers)
    return value

  def policy(self, obs):
    layers = self.shared_layers(obs)
    policy_logits = self.policy_layers(layers)
    return policy_logits

  def forward(self, obs):
    layers = self.shared_layers(obs)
    policy_logits = self.policy_layers(layers)
    value = self.value_layers(layers)
    return policy_logits, value


class PPOTrainer():
  # parameters set based on literature by (Schulman et al., 2017)
  def __init__(self,
               actor_critic,
               ppo_clipping=0.2,
               max_policy_train_iters=80,
               value_train_iters=80,
               policy_lr=3e-4,
               value_lr=1e-2):
    self.ac = actor_critic
    self.ppo_clipping = ppo_clipping
    self.max_policy_train_iters = max_policy_train_iters
    self.value_train_iters = value_train_iters

    policy_params = list(self.ac.shared_layers.parameters()) + \
        list(self.ac.policy_layers.parameters())
    self.policy_optim = optim.Adam(policy_params, lr=policy_lr)

    value_params = list(self.ac.shared_layers.parameters()) + \
        list(self.ac.value_layers.parameters())
    self.value_optim = optim.Adam(value_params, lr=value_lr)
```

```
  def train_policy(self, obs, acts, old_log_probs, gaes):
    for _ in range(self.max_policy_train_iters):
      self.policy_optim.zero_grad()

      new_logits = self.ac.policy(obs)
      new_logits = Categorical(logits=new_logits)
      new_log_probs = new_logits.log_prob(acts)

      policy_ratio = torch.exp(new_log_probs - old_log_probs)

      # policy function ratio of new and old policy must be clipped in range (1-clip,
1+clip)
      clipped_ratio = policy_ratio.clamp(
          1 - self.ppo_clipping, 1 + self.ppo_clipping)

      clipped_loss = clipped_ratio * gaes
      full_loss = policy_ratio * gaes
      policy_loss = -torch.min(full_loss, clipped_loss).mean()

      policy_loss.backward()
      self.policy_optim.step()

  def train_value(self, obs, returns):
    for _ in range(self.value_train_iters):
      self.value_optim.zero_grad()

      values = self.ac.value(obs)
      value_loss = (returns - values) ** 2
      value_loss = value_loss.mean()

      value_loss.backward()
      self.value_optim.step()

def discount_rewards(rewards, gamma=0.99):

    new_rewards = [float(rewards[-1])]
    for i in reversed(range(len(rewards)-1)):
        new_rewards.append(float(rewards[i]) + gamma * new_rewards[-1])
    return np.array(new_rewards[::-1])

def calculate_gaes(rewards, values, gamma=0.99, decay=0.97):
    """
```

```
    calculating the advantage function using the General Advantage Estimation
(GAE) method
    referenced from(Schulman et al., 2018): https://arxiv.org/pdf/1506.02438.pdf
    """
    GAE_param = np.concatenate([values[1:], [0]])
    deltas = [rew + gamma * next_val - val for rew, val, next_val in zip(rewards,
values, GAE_param)]

    gaes = [deltas[-1]]
    for i in reversed(range(len(deltas)-1)):
        gaes.append(deltas[i] + decay * gamma * gaes[-1])

    return np.array(gaes[::-1])

// maximum reward is 1000(1000 time steps)
def rollout(model, env, max_steps=1000):

    ### Create store for data about observation, action, reward, and action value
probability
    train_data = [[], [], [], [], []]
    obs = env.reset()

    obs = obs[0]

    exit
    ep_reward = 0
    for _ in range(max_steps):
        logits, val = model(torch.tensor(obs, dtype=torch.float32,device=DEVICE))

        act_distribution = Categorical(logits=logits)
        act = act_distribution.sample()
        act_log_prob = act_distribution.log_prob(act).item()

        act, val = act.item(), val.item()

        next_obs, reward, done, _, e = env.step(act)

        for i, item in enumerate((obs, act, reward, val, act_log_prob)):
          train_data[i].append(item)

        obs = next_obs
        ep_reward += reward
```

```
        if done:
            break

    train_data = [np.asarray(x) for x in train_data]

    train_data[3] = calculate_gaes(train_data[2], train_data[3])

    return train_data, ep_reward

# memory usage tracking
@profile
def mainFunc():
    env = gym.make('CartPole-v1')

    model = ActorCriticNetwork(env.observation_space.shape[0],
env.action_space.n)
    model = model.to(DEVICE)

    train_data, reward = rollout(model, env)

    #  number of episodes to run for
    n_episodes = 100

    #frequncy of printing reward
    print_freq = 1

    ppo = PPOTrainer(
        model,
        policy_lr = 3e-4,
        value_lr = 1e-3,
        max_policy_train_iters = 40,
        value_train_iters = 40)

    # Training loop
    ep_rewards = []
    for episode_idx in range(n_episodes):
      # Perform rollout
      train_data, reward = rollout(model, env)
      ep_rewards.append(reward)

      # Shuffle
      permute_idxs = np.random.permutation(len(train_data[0]))
```

```
    # Policy data
    obs = torch.tensor(train_data[0][permute_idxs],
                       dtype=torch.float32, device=DEVICE)
    acts = torch.tensor(train_data[1][permute_idxs],
                        dtype=torch.int32, device=DEVICE)
    gaes = torch.tensor(train_data[3][permute_idxs],
                        dtype=torch.float32, device=DEVICE)
    act_log_probs = torch.tensor(train_data[4][permute_idxs],
                                 dtype=torch.float32, device=DEVICE)

    # Value data
    returns = discount_rewards(train_data[2])[permute_idxs]
    returns = torch.tensor(returns, dtype=torch.float32, device=DEVICE)

    # Train model
    ppo.train_policy(obs, acts, act_log_probs, gaes)
    ppo.train_value(obs, returns)

    if (episode_idx + 1) % print_freq == 0:
      print('Episode {} || Avg rewards {:.1f}'.format(
          episode_idx + 1, np.mean(ep_rewards[-print_freq:])))

  et = time.time()
  elasped_time = et - st
  print("Execution time: ", elasped_time, "seconds")

# plot the graphs
  plt.plot(ep_rewards,color='blue',linewidth=1)
  plt.xlabel('Episode')
  plt.ylabel('Sum of Rewards in Episode')
  plt.yscale('log')
  plt.savefig('convergence.png')
  plt.show()
mainFunc()
```

Raw Data

200 Episodes (PPO)

Episode 1 || Avg rewards 13.0
Episode 2 || Avg rewards 13.0
Episode 3 || Avg rewards 12.0
Episode 4 || Avg rewards 8.0
Episode 5 || Avg rewards 14.0
Episode 6 || Avg rewards 12.0
Episode 7 || Avg rewards 14.0
Episode 8 || Avg rewards 15.0
Episode 9 || Avg rewards 10.0
Episode 10 || Avg rewards 12.0
Episode 11 || Avg rewards 10.0
Episode 12 || Avg rewards 11.0
Episode 13 || Avg rewards 12.0
Episode 14 || Avg rewards 9.0
Episode 15 || Avg rewards 13.0
Episode 16 || Avg rewards 10.0
Episode 17 || Avg rewards 10.0
Episode 18 || Avg rewards 14.0
Episode 19 || Avg rewards 11.0
Episode 20 || Avg rewards 10.0
Episode 21 || Avg rewards 10.0
Episode 22 || Avg rewards 9.0
Episode 23 || Avg rewards 12.0
Episode 24 || Avg rewards 9.0
Episode 25 || Avg rewards 9.0
Episode 26 || Avg rewards 13.0
Episode 27 || Avg rewards 10.0
Episode 28 || Avg rewards 10.0
Episode 29 || Avg rewards 11.0
Episode 30 || Avg rewards 11.0
Episode 31 || Avg rewards 12.0
Episode 32 || Avg rewards 14.0
Episode 33 || Avg rewards 10.0
Episode 34 || Avg rewards 14.0
Episode 35 || Avg rewards 9.0
Episode 36 || Avg rewards 14.0
Episode 37 || Avg rewards 15.0
Episode 38 || Avg rewards 10.0
Episode 39 || Avg rewards 11.0

Episode 40 || Avg rewards 9.0
Episode 41 || Avg rewards 13.0
Episode 42 || Avg rewards 11.0
Episode 43 || Avg rewards 10.0
Episode 44 || Avg rewards 10.0
Episode 45 || Avg rewards 10.0
Episode 46 || Avg rewards 9.0
Episode 47 || Avg rewards 12.0
Episode 48 || Avg rewards 10.0
Episode 49 || Avg rewards 12.0
Episode 50 || Avg rewards 12.0
Episode 51 || Avg rewards 14.0
Episode 52 || Avg rewards 10.0
Episode 53 || Avg rewards 13.0
Episode 54 || Avg rewards 12.0
Episode 55 || Avg rewards 13.0
Episode 56 || Avg rewards 12.0
Episode 57 || Avg rewards 14.0
Episode 58 || Avg rewards 12.0
Episode 59 || Avg rewards 16.0
Episode 60 || Avg rewards 12.0
Episode 61 || Avg rewards 13.0
Episode 62 || Avg rewards 13.0
Episode 63 || Avg rewards 10.0
Episode 64 || Avg rewards 15.0
Episode 65 || Avg rewards 12.0
Episode 66 || Avg rewards 11.0
Episode 67 || Avg rewards 11.0
Episode 68 || Avg rewards 11.0
Episode 69 || Avg rewards 12.0
Episode 70 || Avg rewards 11.0
Episode 71 || Avg rewards 11.0
Episode 72 || Avg rewards 11.0
Episode 73 || Avg rewards 11.0
Episode 74 || Avg rewards 12.0
Episode 75 || Avg rewards 11.0
Episode 76 || Avg rewards 11.0
Episode 77 || Avg rewards 9.0
Episode 78 || Avg rewards 12.0
Episode 79 || Avg rewards 11.0
Episode 80 || Avg rewards 11.0
Episode 81 || Avg rewards 14.0
Episode 82 || Avg rewards 13.0

Episode 83 || Avg rewards 10.0
Episode 84 || Avg rewards 11.0
Episode 85 || Avg rewards 11.0
Episode 86 || Avg rewards 12.0
Episode 87 || Avg rewards 10.0
Episode 88 || Avg rewards 15.0
Episode 89 || Avg rewards 10.0
Episode 90 || Avg rewards 13.0
Episode 91 || Avg rewards 13.0
Episode 92 || Avg rewards 12.0
Episode 93 || Avg rewards 11.0
Episode 94 || Avg rewards 17.0
Episode 95 || Avg rewards 17.0
Episode 96 || Avg rewards 20.0
Episode 97 || Avg rewards 13.0
Episode 98 || Avg rewards 19.0
Episode 99 || Avg rewards 18.0
Episode 100 || Avg rewards 17.0
Episode 101 || Avg rewards 21.0
Episode 102 || Avg rewards 23.0
Episode 103 || Avg rewards 15.0
Episode 104 || Avg rewards 20.0
Episode 105 || Avg rewards 21.0
Episode 106 || Avg rewards 19.0
Episode 107 || Avg rewards 34.0
Episode 108 || Avg rewards 26.0
Episode 109 || Avg rewards 44.0
Episode 110 || Avg rewards 112.0
Episode 111 || Avg rewards 88.0
Episode 112 || Avg rewards 87.0
Episode 113 || Avg rewards 51.0
Episode 114 || Avg rewards 29.0
Episode 115 || Avg rewards 65.0
Episode 116 || Avg rewards 39.0
Episode 117 || Avg rewards 27.0
Episode 118 || Avg rewards 22.0
Episode 119 || Avg rewards 38.0
Episode 120 || Avg rewards 22.0
Episode 121 || Avg rewards 33.0
Episode 122 || Avg rewards 31.0
Episode 123 || Avg rewards 54.0
Episode 124 || Avg rewards 38.0
Episode 125 || Avg rewards 51.0

Episode 126 || Avg rewards 66.0
Episode 127 || Avg rewards 48.0
Episode 128 || Avg rewards 52.0
Episode 129 || Avg rewards 83.0
Episode 130 || Avg rewards 79.0
Episode 131 || Avg rewards 83.0
Episode 132 || Avg rewards 73.0
Episode 133 || Avg rewards 85.0
Episode 134 || Avg rewards 72.0
Episode 135 || Avg rewards 55.0
Episode 136 || Avg rewards 57.0
Episode 137 || Avg rewards 33.0
Episode 138 || Avg rewards 60.0
Episode 139 || Avg rewards 71.0
Episode 140 || Avg rewards 35.0
Episode 141 || Avg rewards 43.0
Episode 142 || Avg rewards 39.0
Episode 143 || Avg rewards 51.0
Episode 144 || Avg rewards 61.0
Episode 145 || Avg rewards 54.0
Episode 146 || Avg rewards 56.0
Episode 147 || Avg rewards 36.0
Episode 148 || Avg rewards 40.0
Episode 149 || Avg rewards 24.0
Episode 150 || Avg rewards 60.0
Episode 151 || Avg rewards 27.0
Episode 152 || Avg rewards 31.0
Episode 153 || Avg rewards 46.0
Episode 154 || Avg rewards 57.0
Episode 155 || Avg rewards 102.0
Episode 156 || Avg rewards 89.0
Episode 157 || Avg rewards 83.0
Episode 158 || Avg rewards 67.0
Episode 159 || Avg rewards 109.0
Episode 160 || Avg rewards 83.0
Episode 161 || Avg rewards 96.0
Episode 162 || Avg rewards 115.0
Episode 163 || Avg rewards 109.0
Episode 164 || Avg rewards 107.0
Episode 165 || Avg rewards 143.0
Episode 166 || Avg rewards 123.0
Episode 167 || Avg rewards 137.0
Episode 168 || Avg rewards 169.0

Episode 169 || Avg rewards 108.0
Episode 170 || Avg rewards 220.0
Episode 171 || Avg rewards 228.0
Episode 172 || Avg rewards 261.0
Episode 173 || Avg rewards 240.0
Episode 174 || Avg rewards 250.0
Episode 175 || Avg rewards 284.0
Episode 176 || Avg rewards 202.0
Episode 177 || Avg rewards 215.0
Episode 178 || Avg rewards 323.0
Episode 179 || Avg rewards 213.0
Episode 180 || Avg rewards 153.0
Episode 181 || Avg rewards 536.0
Episode 182 || Avg rewards 269.0
Episode 183 || Avg rewards 220.0
Episode 184 || Avg rewards 173.0
Episode 185 || Avg rewards 166.0
Episode 186 || Avg rewards 145.0
Episode 187 || Avg rewards 184.0
Episode 188 || Avg rewards 250.0
Episode 189 || Avg rewards 233.0
Episode 190 || Avg rewards 239.0
Episode 191 || Avg rewards 473.0
Episode 192 || Avg rewards 666.0
Episode 193 || Avg rewards 297.0
Episode 194 || Avg rewards 735.0
Episode 195 || Avg rewards 230.0
Episode 196 || Avg rewards 295.0
Episode 197 || Avg rewards 141.0
Episode 198 || Avg rewards 181.0
Episode 199 || Avg rewards 95.0
Episode 200 || Avg rewards 145.0

1000 Episodes (PPO)

Episode 1 || Avg rewards 12.0
Episode 2 || Avg rewards 10.0
Episode 3 || Avg rewards 8.0
Episode 4 || Avg rewards 9.0
Episode 5 || Avg rewards 10.0
Episode 6 || Avg rewards 10.0
Episode 7 || Avg rewards 9.0
Episode 8 || Avg rewards 9.0
Episode 9 || Avg rewards 10.0
Episode 10 || Avg rewards 9.0
Episode 11 || Avg rewards 10.0
Episode 12 || Avg rewards 11.0
Episode 13 || Avg rewards 11.0
Episode 14 || Avg rewards 9.0
Episode 15 || Avg rewards 9.0
Episode 16 || Avg rewards 9.0
Episode 17 || Avg rewards 10.0
Episode 18 || Avg rewards 8.0
Episode 19 || Avg rewards 11.0
Episode 20 || Avg rewards 14.0
Episode 21 || Avg rewards 9.0
Episode 22 || Avg rewards 9.0
Episode 23 || Avg rewards 12.0
Episode 24 || Avg rewards 12.0
Episode 25 || Avg rewards 13.0
Episode 26 || Avg rewards 12.0
Episode 27 || Avg rewards 9.0
Episode 28 || Avg rewards 11.0
Episode 29 || Avg rewards 10.0
Episode 30 || Avg rewards 10.0
Episode 31 || Avg rewards 11.0
Episode 32 || Avg rewards 11.0
Episode 33 || Avg rewards 12.0
Episode 34 || Avg rewards 10.0
Episode 35 || Avg rewards 9.0
Episode 36 || Avg rewards 11.0
Episode 37 || Avg rewards 12.0
Episode 38 || Avg rewards 13.0
Episode 39 || Avg rewards 10.0
Episode 40 || Avg rewards 14.0
Episode 41 || Avg rewards 12.0

Episode 42 || Avg rewards 9.0
Episode 43 || Avg rewards 10.0
Episode 44 || Avg rewards 11.0
Episode 45 || Avg rewards 11.0
Episode 46 || Avg rewards 12.0
Episode 47 || Avg rewards 11.0
Episode 48 || Avg rewards 9.0
Episode 49 || Avg rewards 12.0
Episode 50 || Avg rewards 12.0
Episode 51 || Avg rewards 10.0
Episode 52 || Avg rewards 10.0
Episode 53 || Avg rewards 10.0
Episode 54 || Avg rewards 11.0
Episode 55 || Avg rewards 10.0
Episode 56 || Avg rewards 10.0
Episode 57 || Avg rewards 10.0
Episode 58 || Avg rewards 10.0
Episode 59 || Avg rewards 10.0
Episode 60 || Avg rewards 9.0
Episode 61 || Avg rewards 12.0
Episode 62 || Avg rewards 10.0
Episode 63 || Avg rewards 11.0
Episode 64 || Avg rewards 11.0
Episode 65 || Avg rewards 12.0
Episode 66 || Avg rewards 9.0
Episode 67 || Avg rewards 11.0
Episode 68 || Avg rewards 8.0
Episode 69 || Avg rewards 10.0
Episode 70 || Avg rewards 11.0
Episode 71 || Avg rewards 10.0
Episode 72 || Avg rewards 15.0
Episode 73 || Avg rewards 11.0
Episode 74 || Avg rewards 17.0
Episode 75 || Avg rewards 15.0
Episode 76 || Avg rewards 16.0
Episode 77 || Avg rewards 16.0
Episode 78 || Avg rewards 22.0
Episode 79 || Avg rewards 15.0
Episode 80 || Avg rewards 23.0
Episode 81 || Avg rewards 67.0
Episode 82 || Avg rewards 83.0
Episode 83 || Avg rewards 49.0
Episode 84 || Avg rewards 88.0

Episode 85 || Avg rewards 234.0
Episode 86 || Avg rewards 148.0
Episode 87 || Avg rewards 61.0
Episode 88 || Avg rewards 75.0
Episode 89 || Avg rewards 79.0
Episode 90 || Avg rewards 106.0
Episode 91 || Avg rewards 72.0
Episode 92 || Avg rewards 53.0
Episode 93 || Avg rewards 90.0
Episode 94 || Avg rewards 52.0
Episode 95 || Avg rewards 83.0
Episode 96 || Avg rewards 92.0
Episode 97 || Avg rewards 76.0
Episode 98 || Avg rewards 105.0
Episode 99 || Avg rewards 143.0
Episode 100 || Avg rewards 91.0
Episode 101 || Avg rewards 146.0
Episode 102 || Avg rewards 135.0
Episode 103 || Avg rewards 135.0
Episode 104 || Avg rewards 149.0
Episode 105 || Avg rewards 177.0
Episode 106 || Avg rewards 206.0
Episode 107 || Avg rewards 190.0
Episode 108 || Avg rewards 165.0

.
. (Due to limited space I have curtailed the data to 5 pages for each trial)
.

Episode 900 || Avg rewards 59.0
Episode 901 || Avg rewards 86.0
Episode 902 || Avg rewards 73.0
Episode 903 || Avg rewards 73.0
Episode 904 || Avg rewards 75.0
Episode 905 || Avg rewards 91.0
Episode 906 || Avg rewards 78.0
Episode 907 || Avg rewards 72.0
Episode 908 || Avg rewards 90.0
Episode 909 || Avg rewards 89.0
Episode 910 || Avg rewards 65.0
Episode 911 || Avg rewards 77.0
Episode 912 || Avg rewards 62.0
Episode 913 || Avg rewards 72.0
Episode 914 || Avg rewards 72.0

Episode 915 || Avg rewards 79.0
Episode 916 || Avg rewards 61.0
Episode 917 || Avg rewards 60.0
Episode 918 || Avg rewards 72.0
Episode 919 || Avg rewards 60.0
Episode 920 || Avg rewards 73.0
Episode 921 || Avg rewards 73.0
Episode 922 || Avg rewards 78.0
Episode 923 || Avg rewards 71.0
Episode 924 || Avg rewards 72.0
Episode 925 || Avg rewards 72.0
Episode 926 || Avg rewards 81.0
Episode 927 || Avg rewards 85.0
Episode 928 || Avg rewards 76.0
Episode 929 || Avg rewards 88.0
Episode 930 || Avg rewards 98.0
Episode 931 || Avg rewards 93.0
Episode 932 || Avg rewards 87.0
Episode 933 || Avg rewards 66.0
Episode 934 || Avg rewards 80.0
Episode 935 || Avg rewards 94.0
Episode 936 || Avg rewards 83.0
Episode 937 || Avg rewards 93.0
Episode 938 || Avg rewards 73.0
Episode 939 || Avg rewards 66.0
Episode 940 || Avg rewards 90.0
Episode 941 || Avg rewards 113.0
Episode 942 || Avg rewards 95.0
Episode 943 || Avg rewards 113.0
Episode 944 || Avg rewards 104.0
Episode 945 || Avg rewards 99.0
Episode 946 || Avg rewards 97.0
Episode 947 || Avg rewards 105.0
Episode 948 || Avg rewards 105.0
Episode 949 || Avg rewards 122.0
Episode 950 || Avg rewards 106.0
Episode 951 || Avg rewards 107.0
Episode 952 || Avg rewards 99.0
Episode 953 || Avg rewards 104.0
Episode 954 || Avg rewards 109.0
Episode 955 || Avg rewards 117.0
Episode 956 || Avg rewards 125.0
Episode 957 || Avg rewards 127.0

Episode 958 || Avg rewards 101.0
Episode 959 || Avg rewards 102.0
Episode 960 || Avg rewards 121.0
Episode 961 || Avg rewards 122.0
Episode 962 || Avg rewards 141.0
Episode 963 || Avg rewards 137.0
Episode 964 || Avg rewards 129.0
Episode 965 || Avg rewards 186.0
Episode 966 || Avg rewards 145.0
Episode 967 || Avg rewards 118.0
Episode 968 || Avg rewards 143.0
Episode 969 || Avg rewards 191.0
Episode 970 || Avg rewards 163.0
Episode 971 || Avg rewards 181.0
Episode 972 || Avg rewards 283.0
Episode 973 || Avg rewards 222.0
Episode 974 || Avg rewards 501.0
Episode 975 || Avg rewards 386.0
Episode 976 || Avg rewards 452.0
Episode 977 || Avg rewards 420.0
Episode 978 || Avg rewards 359.0
Episode 979 || Avg rewards 445.0
Episode 980 || Avg rewards 429.0
Episode 981 || Avg rewards 759.0
Episode 982 || Avg rewards 560.0
Episode 983 || Avg rewards 948.0
Episode 984 || Avg rewards 616.0
Episode 985 || Avg rewards 105.0
Episode 986 || Avg rewards 1000.0
Episode 987 || Avg rewards 1000.0
Episode 988 || Avg rewards 1000.0
Episode 989 || Avg rewards 1000.0
Episode 990 || Avg rewards 1000.0
Episode 991 || Avg rewards 1000.0
Episode 992 || Avg rewards 1000.0
Episode 993 || Avg rewards 1000.0
Episode 994 || Avg rewards 1000.0
Episode 995 || Avg rewards 1000.0
Episode 996 || Avg rewards 1000.0
Episode 997 || Avg rewards 1000.0
Episode 998 || Avg rewards 1000.0
Episode 999 || Avg rewards 1000.0
Episode 1000 || Avg rewards 1000.0

3000 Episodes (PPO)

Episode 1 || Avg rewards 16.0
Episode 2 || Avg rewards 9.0
Episode 3 || Avg rewards 9.0
Episode 4 || Avg rewards 10.0
Episode 5 || Avg rewards 9.0
Episode 6 || Avg rewards 10.0
Episode 7 || Avg rewards 11.0
Episode 8 || Avg rewards 9.0
Episode 9 || Avg rewards 12.0
Episode 10 || Avg rewards 12.0
Episode 11 || Avg rewards 11.0
Episode 12 || Avg rewards 12.0
Episode 13 || Avg rewards 9.0
Episode 14 || Avg rewards 12.0
Episode 15 || Avg rewards 8.0
Episode 16 || Avg rewards 10.0
Episode 17 || Avg rewards 9.0
Episode 18 || Avg rewards 10.0
Episode 19 || Avg rewards 11.0
Episode 20 || Avg rewards 10.0
Episode 21 || Avg rewards 13.0
Episode 22 || Avg rewards 11.0
Episode 23 || Avg rewards 13.0
Episode 24 || Avg rewards 12.0
Episode 25 || Avg rewards 11.0
Episode 26 || Avg rewards 10.0
Episode 27 || Avg rewards 10.0
Episode 28 || Avg rewards 11.0
Episode 29 || Avg rewards 10.0
Episode 30 || Avg rewards 13.0
Episode 31 || Avg rewards 11.0
Episode 32 || Avg rewards 11.0
Episode 33 || Avg rewards 15.0
Episode 34 || Avg rewards 14.0
Episode 35 || Avg rewards 10.0
Episode 36 || Avg rewards 9.0
Episode 37 || Avg rewards 10.0
Episode 38 || Avg rewards 10.0
Episode 39 || Avg rewards 10.0
Episode 40 || Avg rewards 9.0
Episode 41 || Avg rewards 11.0

Episode 42 || Avg rewards 10.0
Episode 43 || Avg rewards 11.0
Episode 44 || Avg rewards 10.0
Episode 45 || Avg rewards 9.0
Episode 46 || Avg rewards 12.0
Episode 47 || Avg rewards 9.0
Episode 48 || Avg rewards 8.0
Episode 49 || Avg rewards 9.0
Episode 50 || Avg rewards 10.0
Episode 51 || Avg rewards 12.0
Episode 52 || Avg rewards 9.0
Episode 53 || Avg rewards 12.0
Episode 54 || Avg rewards 10.0
Episode 55 || Avg rewards 10.0
Episode 56 || Avg rewards 14.0
Episode 57 || Avg rewards 11.0
Episode 58 || Avg rewards 14.0
Episode 59 || Avg rewards 9.0
Episode 60 || Avg rewards 17.0
Episode 61 || Avg rewards 17.0
Episode 62 || Avg rewards 30.0
Episode 63 || Avg rewards 42.0
Episode 64 || Avg rewards 42.0
Episode 65 || Avg rewards 25.0
Episode 66 || Avg rewards 56.0
Episode 67 || Avg rewards 82.0
Episode 68 || Avg rewards 78.0
Episode 69 || Avg rewards 98.0
Episode 70 || Avg rewards 83.0
Episode 71 || Avg rewards 78.0
Episode 72 || Avg rewards 170.0
Episode 73 || Avg rewards 69.0
Episode 74 || Avg rewards 105.0
Episode 75 || Avg rewards 104.0
Episode 76 || Avg rewards 68.0
Episode 77 || Avg rewards 76.0
Episode 78 || Avg rewards 150.0
Episode 79 || Avg rewards 97.0
Episode 80 || Avg rewards 66.0
Episode 81 || Avg rewards 71.0
Episode 82 || Avg rewards 114.0
Episode 83 || Avg rewards 91.0
Episode 84 || Avg rewards 86.0

Episode 85 || Avg rewards 106.0
Episode 86 || Avg rewards 71.0
Episode 87 || Avg rewards 77.0
Episode 88 || Avg rewards 60.0
Episode 89 || Avg rewards 64.0
Episode 90 || Avg rewards 82.0
Episode 91 || Avg rewards 97.0
Episode 92 || Avg rewards 76.0
Episode 93 || Avg rewards 92.0
Episode 94 || Avg rewards 138.0
Episode 95 || Avg rewards 172.0
Episode 96 || Avg rewards 162.0
Episode 97 || Avg rewards 289.0
Episode 98 || Avg rewards 151.0
Episode 99 || Avg rewards 233.0
Episode 100 || Avg rewards 206.0
Episode 101 || Avg rewards 198.0
Episode 102 || Avg rewards 154.0
Episode 103 || Avg rewards 170.0
Episode 104 || Avg rewards 205.0
Episode 105 || Avg rewards 229.0
Episode 106 || Avg rewards 196.0
Episode 107 || Avg rewards 153.0
Episode 108 || Avg rewards 160.0
Episode 109 || Avg rewards 208.0
Episode 110 || Avg rewards 202.0
Episode 111 || Avg rewards 224.0
Episode 112 || Avg rewards 151.0
Episode 113 || Avg rewards 253.0
Episode 114 || Avg rewards 451.0

.
. (Due to limited space I have curtailed the data to 5 pages for each trial)
.

Episode 2900 || Avg rewards 1000.0
Episode 2901 || Avg rewards 1000.0
Episode 2902 || Avg rewards 1000.0
Episode 2903 || Avg rewards 1000.0
Episode 2904 || Avg rewards 1000.0
Episode 2905 || Avg rewards 1000.0
Episode 2906 || Avg rewards 1000.0
Episode 2907 || Avg rewards 1000.0
Episode 2908 || Avg rewards 1000.0

Episode 2909 || Avg rewards 1000.0
Episode 2910 || Avg rewards 1000.0
Episode 2911 || Avg rewards 1000.0
Episode 2912 || Avg rewards 1000.0
Episode 2913 || Avg rewards 1000.0
Episode 2914 || Avg rewards 1000.0
Episode 2915 || Avg rewards 1000.0
Episode 2916 || Avg rewards 1000.0
Episode 2917 || Avg rewards 1000.0
Episode 2918 || Avg rewards 1000.0
Episode 2919 || Avg rewards 1000.0
Episode 2920 || Avg rewards 1000.0
Episode 2921 || Avg rewards 1000.0
Episode 2922 || Avg rewards 1000.0
Episode 2923 || Avg rewards 1000.0
Episode 2924 || Avg rewards 1000.0
Episode 2925 || Avg rewards 1000.0
Episode 2926 || Avg rewards 1000.0
Episode 2927 || Avg rewards 1000.0
Episode 2928 || Avg rewards 1000.0
Episode 2929 || Avg rewards 1000.0
Episode 2930 || Avg rewards 1000.0
Episode 2931 || Avg rewards 1000.0
Episode 2932 || Avg rewards 1000.0
Episode 2933 || Avg rewards 1000.0
Episode 2934 || Avg rewards 1000.0
Episode 2935 || Avg rewards 1000.0
Episode 2936 || Avg rewards 1000.0
Episode 2937 || Avg rewards 1000.0
Episode 2938 || Avg rewards 1000.0
Episode 2939 || Avg rewards 1000.0
Episode 2940 || Avg rewards 1000.0
Episode 2941 || Avg rewards 1000.0
Episode 2942 || Avg rewards 1000.0
Episode 2943 || Avg rewards 1000.0
Episode 2944 || Avg rewards 1000.0
Episode 2945 || Avg rewards 1000.0
Episode 2946 || Avg rewards 1000.0
Episode 2947 || Avg rewards 1000.0
Episode 2948 || Avg rewards 1000.0
Episode 2949 || Avg rewards 1000.0
Episode 2950 || Avg rewards 1000.0
Episode 2951 || Avg rewards 1000.0

Episode 2952 || Avg rewards 1000.0
Episode 2953 || Avg rewards 1000.0
Episode 2954 || Avg rewards 1000.0
Episode 2955 || Avg rewards 1000.0
Episode 2956 || Avg rewards 1000.0
Episode 2957 || Avg rewards 1000.0
Episode 2958 || Avg rewards 1000.0
Episode 2959 || Avg rewards 1000.0
Episode 2960 || Avg rewards 1000.0
Episode 2961 || Avg rewards 1000.0
Episode 2962 || Avg rewards 1000.0
Episode 2963 || Avg rewards 1000.0
Episode 2964 || Avg rewards 1000.0
Episode 2965 || Avg rewards 1000.0
Episode 2966 || Avg rewards 1000.0
Episode 2967 || Avg rewards 1000.0
Episode 2968 || Avg rewards 1000.0
Episode 2969 || Avg rewards 1000.0
Episode 2970 || Avg rewards 1000.0
Episode 2971 || Avg rewards 1000.0
Episode 2972 || Avg rewards 1000.0
Episode 2973 || Avg rewards 1000.0
Episode 2974 || Avg rewards 1000.0
Episode 2975 || Avg rewards 1000.0
Episode 2976 || Avg rewards 1000.0
Episode 2977 || Avg rewards 1000.0
Episode 2978 || Avg rewards 1000.0
Episode 2979 || Avg rewards 1000.0
Episode 2980 || Avg rewards 1000.0
Episode 2981 || Avg rewards 1000.0
Episode 2982 || Avg rewards 1000.0
Episode 2983 || Avg rewards 1000.0
Episode 2984 || Avg rewards 1000.0
Episode 2985 || Avg rewards 1000.0
Episode 2986 || Avg rewards 1000.0
Episode 2987 || Avg rewards 1000.0
Episode 2988 || Avg rewards 1000.0
Episode 2989 || Avg rewards 1000.0
Episode 2990 || Avg rewards 1000.0
Episode 2991 || Avg rewards 1000.0
Episode 2992 || Avg rewards 1000.0
Episode 2993 || Avg rewards 1000.0
Episode 2994 || Avg rewards 1000.0

Episode 2995 || Avg rewards 1000.0
Episode 2996 || Avg rewards 1000.0
Episode 2997 || Avg rewards 1000.0
Episode 2998 || Avg rewards 1000.0
Episode 2999 || Avg rewards 1000.0

200 Episodes (Q Learning)

Episode 0 || Avg rewards 12.0
Episode 1 || Avg rewards 17.0
Episode 2 || Avg rewards 12.0
Episode 3 || Avg rewards 10.0
Episode 4 || Avg rewards 19.0
Episode 5 || Avg rewards 20.0
Episode 6 || Avg rewards 36.0
Episode 7 || Avg rewards 20.0
Episode 8 || Avg rewards 17.0
Episode 9 || Avg rewards 53.0
Episode 10 || Avg rewards 16.0
Episode 11 || Avg rewards 43.0
Episode 12 || Avg rewards 13.0
Episode 13 || Avg rewards 16.0
Episode 14 || Avg rewards 18.0
Episode 15 || Avg rewards 24.0
Episode 16 || Avg rewards 19.0
Episode 17 || Avg rewards 20.0
Episode 18 || Avg rewards 37.0
Episode 19 || Avg rewards 17.0
Episode 20 || Avg rewards 17.0
Episode 21 || Avg rewards 28.0
Episode 22 || Avg rewards 13.0
Episode 23 || Avg rewards 31.0
Episode 24 || Avg rewards 18.0
Episode 25 || Avg rewards 37.0
Episode 26 || Avg rewards 21.0
Episode 27 || Avg rewards 10.0
Episode 28 || Avg rewards 15.0
Episode 29 || Avg rewards 18.0
Episode 30 || Avg rewards 21.0
Episode 31 || Avg rewards 31.0
Episode 32 || Avg rewards 13.0
Episode 33 || Avg rewards 20.0

Episode 34 || Avg rewards 21.0
Episode 35 || Avg rewards 14.0
Episode 36 || Avg rewards 28.0
Episode 37 || Avg rewards 10.0
Episode 38 || Avg rewards 37.0
Episode 39 || Avg rewards 44.0
Episode 40 || Avg rewards 31.0
Episode 41 || Avg rewards 18.0
Episode 42 || Avg rewards 27.0
Episode 43 || Avg rewards 14.0
Episode 44 || Avg rewards 18.0
Episode 45 || Avg rewards 32.0
Episode 46 || Avg rewards 20.0
Episode 47 || Avg rewards 24.0
Episode 48 || Avg rewards 18.0
Episode 49 || Avg rewards 14.0
Episode 50 || Avg rewards 30.0
Episode 51 || Avg rewards 21.0
Episode 52 || Avg rewards 30.0
Episode 53 || Avg rewards 58.0
Episode 54 || Avg rewards 46.0
Episode 55 || Avg rewards 10.0
Episode 56 || Avg rewards 14.0
Episode 57 || Avg rewards 11.0
Episode 58 || Avg rewards 10.0
Episode 59 || Avg rewards 18.0
Episode 60 || Avg rewards 74.0
Episode 61 || Avg rewards 21.0
Episode 62 || Avg rewards 14.0
Episode 63 || Avg rewards 44.0
Episode 64 || Avg rewards 11.0
Episode 65 || Avg rewards 16.0
Episode 66 || Avg rewards 13.0
Episode 67 || Avg rewards 39.0
Episode 68 || Avg rewards 23.0
Episode 69 || Avg rewards 12.0
Episode 70 || Avg rewards 9.0
Episode 71 || Avg rewards 47.0
Episode 72 || Avg rewards 21.0
Episode 73 || Avg rewards 28.0
Episode 74 || Avg rewards 17.0
Episode 75 || Avg rewards 21.0
Episode 76 || Avg rewards 32.0

Episode 77 || Avg rewards 25.0
Episode 78 || Avg rewards 29.0
Episode 79 || Avg rewards 12.0
Episode 80 || Avg rewards 18.0
Episode 81 || Avg rewards 10.0
Episode 82 || Avg rewards 14.0
Episode 83 || Avg rewards 44.0
Episode 84 || Avg rewards 18.0
Episode 85 || Avg rewards 33.0
Episode 86 || Avg rewards 18.0
Episode 87 || Avg rewards 15.0
Episode 88 || Avg rewards 11.0
Episode 89 || Avg rewards 18.0
Episode 90 || Avg rewards 28.0
Episode 91 || Avg rewards 23.0
Episode 92 || Avg rewards 14.0
Episode 93 || Avg rewards 10.0
Episode 94 || Avg rewards 27.0
Episode 95 || Avg rewards 11.0
Episode 96 || Avg rewards 29.0
Episode 97 || Avg rewards 11.0
Episode 98 || Avg rewards 16.0
Episode 99 || Avg rewards 16.0
Episode 100 || Avg rewards 91.0
Episode 101 || Avg rewards 88.0
Episode 102 || Avg rewards 20.0
Episode 103 || Avg rewards 73.0
Episode 104 || Avg rewards 57.0
Episode 105 || Avg rewards 32.0
Episode 106 || Avg rewards 77.0
Episode 107 || Avg rewards 35.0
Episode 108 || Avg rewards 129.0
Episode 109 || Avg rewards 61.0
Episode 110 || Avg rewards 41.0
Episode 111 || Avg rewards 57.0
Episode 112 || Avg rewards 32.0
Episode 113 || Avg rewards 72.0
Episode 114 || Avg rewards 55.0
Episode 115 || Avg rewards 41.0
Episode 116 || Avg rewards 86.0
Episode 117 || Avg rewards 50.0
Episode 118 || Avg rewards 57.0
Episode 119 || Avg rewards 63.0

Episode 120 || Avg rewards 64.0
Episode 121 || Avg rewards 105.0
Episode 122 || Avg rewards 47.0
Episode 123 || Avg rewards 45.0
Episode 124 || Avg rewards 72.0
Episode 125 || Avg rewards 11.0
Episode 126 || Avg rewards 57.0
Episode 127 || Avg rewards 48.0
Episode 128 || Avg rewards 42.0
Episode 129 || Avg rewards 55.0
Episode 130 || Avg rewards 47.0
Episode 131 || Avg rewards 49.0
Episode 132 || Avg rewards 73.0
Episode 133 || Avg rewards 47.0
Episode 134 || Avg rewards 79.0
Episode 135 || Avg rewards 55.0
Episode 136 || Avg rewards 57.0
Episode 137 || Avg rewards 45.0
Episode 138 || Avg rewards 61.0
Episode 139 || Avg rewards 83.0
Episode 140 || Avg rewards 39.0
Episode 141 || Avg rewards 47.0
Episode 142 || Avg rewards 75.0
Episode 143 || Avg rewards 15.0
Episode 144 || Avg rewards 71.0
Episode 145 || Avg rewards 47.0
Episode 146 || Avg rewards 177.0
Episode 147 || Avg rewards 16.0
Episode 148 || Avg rewards 63.0
Episode 149 || Avg rewards 80.0
Episode 150 || Avg rewards 47.0
Episode 151 || Avg rewards 89.0
Episode 152 || Avg rewards 16.0
Episode 153 || Avg rewards 48.0
Episode 154 || Avg rewards 67.0
Episode 155 || Avg rewards 65.0
Episode 156 || Avg rewards 33.0
Episode 157 || Avg rewards 43.0
Episode 158 || Avg rewards 18.0
Episode 159 || Avg rewards 59.0
Episode 160 || Avg rewards 93.0
Episode 161 || Avg rewards 72.0
Episode 162 || Avg rewards 62.0

Episode 163 || Avg rewards 51.0
Episode 164 || Avg rewards 50.0
Episode 165 || Avg rewards 43.0
Episode 166 || Avg rewards 33.0
Episode 167 || Avg rewards 93.0
Episode 168 || Avg rewards 45.0
Episode 169 || Avg rewards 47.0
Episode 170 || Avg rewards 49.0
Episode 171 || Avg rewards 37.0
Episode 172 || Avg rewards 33.0
Episode 173 || Avg rewards 141.0
Episode 174 || Avg rewards 32.0
Episode 175 || Avg rewards 55.0
Episode 176 || Avg rewards 96.0
Episode 177 || Avg rewards 54.0
Episode 178 || Avg rewards 59.0
Episode 179 || Avg rewards 34.0
Episode 180 || Avg rewards 59.0
Episode 181 || Avg rewards 52.0
Episode 182 || Avg rewards 31.0
Episode 183 || Avg rewards 46.0
Episode 184 || Avg rewards 58.0
Episode 185 || Avg rewards 116.0
Episode 186 || Avg rewards 99.0
Episode 187 || Avg rewards 95.0
Episode 188 || Avg rewards 30.0
Episode 189 || Avg rewards 57.0
Episode 190 || Avg rewards 57.0
Episode 191 || Avg rewards 47.0
Episode 192 || Avg rewards 51.0
Episode 193 || Avg rewards 33.0
Episode 194 || Avg rewards 81.0
Episode 195 || Avg rewards 123.0
Episode 196 || Avg rewards 61.0
Episode 197 || Avg rewards 45.0
Episode 198 || Avg rewards 40.0
Episode 199 || Avg rewards 15.0

1000 Episodes (Q Learning)

(Data started from episode 40, as my device could not display the previous episodes due to sheet amount of raw data)
Episode 40 || Avg rewards 27.0
Episode 41 || Avg rewards 22.0
Episode 42 || Avg rewards 18.0
Episode 43 || Avg rewards 12.0
Episode 44 || Avg rewards 12.0
Episode 45 || Avg rewards 31.0
Episode 46 || Avg rewards 13.0
Episode 47 || Avg rewards 21.0
Episode 48 || Avg rewards 14.0
Episode 49 || Avg rewards 24.0
Episode 50 || Avg rewards 15.0
Episode 51 || Avg rewards 19.0
Episode 52 || Avg rewards 22.0
Episode 53 || Avg rewards 13.0
Episode 54 || Avg rewards 16.0
Episode 55 || Avg rewards 16.0
Episode 56 || Avg rewards 27.0
Episode 57 || Avg rewards 17.0
Episode 58 || Avg rewards 11.0
Episode 59 || Avg rewards 28.0
Episode 60 || Avg rewards 25.0
Episode 61 || Avg rewards 28.0
Episode 62 || Avg rewards 26.0
Episode 63 || Avg rewards 18.0
Episode 64 || Avg rewards 27.0
Episode 65 || Avg rewards 25.0
Episode 66 || Avg rewards 13.0
Episode 67 || Avg rewards 16.0
Episode 68 || Avg rewards 20.0
Episode 69 || Avg rewards 36.0
Episode 70 || Avg rewards 16.0
Episode 71 || Avg rewards 14.0
Episode 72 || Avg rewards 17.0
Episode 73 || Avg rewards 29.0
Episode 74 || Avg rewards 39.0
Episode 75 || Avg rewards 20.0
Episode 76 || Avg rewards 14.0
Episode 77 || Avg rewards 22.0
Episode 78 || Avg rewards 14.0

Episode 79 || Avg rewards 15.0
Episode 80 || Avg rewards 13.0
Episode 81 || Avg rewards 27.0
Episode 82 || Avg rewards 66.0
Episode 83 || Avg rewards 51.0
Episode 84 || Avg rewards 23.0
Episode 85 || Avg rewards 13.0
Episode 86 || Avg rewards 11.0
Episode 87 || Avg rewards 28.0
Episode 88 || Avg rewards 45.0
Episode 89 || Avg rewards 11.0
Episode 90 || Avg rewards 27.0
Episode 91 || Avg rewards 25.0
Episode 92 || Avg rewards 18.0
Episode 93 || Avg rewards 16.0
Episode 94 || Avg rewards 19.0
Episode 95 || Avg rewards 15.0
Episode 96 || Avg rewards 59.0
Episode 97 || Avg rewards 18.0
Episode 98 || Avg rewards 15.0
Episode 99 || Avg rewards 38.0
Episode 100 || Avg rewards 59.0
Episode 101 || Avg rewards 81.0
Episode 102 || Avg rewards 35.0
Episode 103 || Avg rewards 31.0
Episode 104 || Avg rewards 90.0
Episode 105 || Avg rewards 40.0
Episode 106 || Avg rewards 56.0
Episode 107 || Avg rewards 57.0
Episode 108 || Avg rewards 41.0
Episode 109 || Avg rewards 127.0
Episode 110 || Avg rewards 112.0
Episode 111 || Avg rewards 39.0
Episode 112 || Avg rewards 45.0
Episode 113 || Avg rewards 93.0
Episode 114 || Avg rewards 51.0
Episode 115 || Avg rewards 78.0
Episode 116 || Avg rewards 25.0
Episode 117 || Avg rewards 44.0
Episode 118 || Avg rewards 46.0
Episode 119 || Avg rewards 63.0
Episode 120 || Avg rewards 49.0
Episode 121 || Avg rewards 89.0

Episode 122 || Avg rewards 64.0
Episode 123 || Avg rewards 75.0
Episode 124 || Avg rewards 50.0
Episode 125 || Avg rewards 47.0
Episode 126 || Avg rewards 39.0
Episode 127 || Avg rewards 125.0
Episode 128 || Avg rewards 41.0
Episode 129 || Avg rewards 82.0
Episode 130 || Avg rewards 43.0
Episode 131 || Avg rewards 66.0
Episode 132 || Avg rewards 51.0
Episode 133 || Avg rewards 44.0
Episode 134 || Avg rewards 50.0
Episode 135 || Avg rewards 90.0
Episode 136 || Avg rewards 53.0
Episode 137 || Avg rewards 48.0
Episode 138 || Avg rewards 43.0
Episode 139 || Avg rewards 89.0
Episode 140 || Avg rewards 59.0
Episode 141 || Avg rewards 54.0
Episode 142 || Avg rewards 52.0
Episode 143 || Avg rewards 31.0
Episode 144 || Avg rewards 86.0
Episode 145 || Avg rewards 57.0
Episode 146 || Avg rewards 54.0
Episode 147 || Avg rewards 65.0
Episode 148 || Avg rewards 123.0
Episode 149 || Avg rewards 74.0
Episode 150 || Avg rewards 59.0
Episode 151 || Avg rewards 63.0
Episode 152 || Avg rewards 43.0
Episode 153 || Avg rewards 49.0
Episode 154 || Avg rewards 51.0
Episode 155 || Avg rewards 53.0
Episode 156 || Avg rewards 73.0
Episode 157 || Avg rewards 90.0
Episode 158 || Avg rewards 61.0
Episode 159 || Avg rewards 94.0
Episode 160 || Avg rewards 61.0
Episode 161 || Avg rewards 45.0
Episode 162 || Avg rewards 53.0
Episode 163 || Avg rewards 40.0
Episode 164 || Avg rewards 53.0

Episode 165 || Avg rewards 42.0
Episode 166 || Avg rewards 49.0
Episode 167 || Avg rewards 51.0
Episode 168 || Avg rewards 67.0
Episode 169 || Avg rewards 53.0

.
. (Due to limited space I have curtailed the data to 5 pages for each trial)
.

Episode 918 || Avg rewards 44.0
Episode 919 || Avg rewards 51.0
Episode 920 || Avg rewards 57.0
Episode 921 || Avg rewards 94.0
Episode 922 || Avg rewards 36.0
Episode 923 || Avg rewards 67.0
Episode 924 || Avg rewards 95.0
Episode 925 || Avg rewards 65.0
Episode 926 || Avg rewards 71.0
Episode 927 || Avg rewards 64.0
Episode 928 || Avg rewards 104.0
Episode 929 || Avg rewards 41.0
Episode 930 || Avg rewards 46.0
Episode 931 || Avg rewards 46.0
Episode 932 || Avg rewards 79.0
Episode 933 || Avg rewards 51.0
Episode 934 || Avg rewards 72.0
Episode 935 || Avg rewards 90.0
Episode 936 || Avg rewards 39.0
Episode 937 || Avg rewards 49.0
Episode 938 || Avg rewards 80.0
Episode 939 || Avg rewards 65.0
Episode 940 || Avg rewards 96.0
Episode 941 || Avg rewards 111.0
Episode 942 || Avg rewards 45.0
Episode 943 || Avg rewards 41.0
Episode 944 || Avg rewards 47.0
Episode 945 || Avg rewards 45.0
Episode 946 || Avg rewards 74.0
Episode 947 || Avg rewards 80.0
Episode 948 || Avg rewards 43.0
Episode 949 || Avg rewards 53.0
Episode 950 || Avg rewards 46.0
Episode 951 || Avg rewards 42.0

Episode 952 || Avg rewards 49.0
Episode 953 || Avg rewards 59.0
Episode 954 || Avg rewards 84.0
Episode 955 || Avg rewards 42.0
Episode 956 || Avg rewards 61.0
Episode 957 || Avg rewards 71.0
Episode 958 || Avg rewards 47.0
Episode 959 || Avg rewards 72.0
Episode 960 || Avg rewards 51.0
Episode 961 || Avg rewards 51.0
Episode 962 || Avg rewards 33.0
Episode 963 || Avg rewards 53.0
Episode 964 || Avg rewards 71.0
Episode 965 || Avg rewards 44.0
Episode 966 || Avg rewards 34.0
Episode 967 || Avg rewards 42.0
Episode 968 || Avg rewards 48.0
Episode 969 || Avg rewards 57.0
Episode 970 || Avg rewards 118.0
Episode 971 || Avg rewards 90.0
Episode 972 || Avg rewards 52.0
Episode 973 || Avg rewards 92.0
Episode 974 || Avg rewards 53.0
Episode 975 || Avg rewards 76.0
Episode 976 || Avg rewards 85.0
Episode 977 || Avg rewards 76.0
Episode 978 || Avg rewards 77.0
Episode 979 || Avg rewards 80.0
Episode 980 || Avg rewards 41.0
Episode 981 || Avg rewards 109.0
Episode 982 || Avg rewards 72.0
Episode 983 || Avg rewards 96.0
Episode 984 || Avg rewards 51.0
Episode 985 || Avg rewards 94.0
Episode 986 || Avg rewards 41.0
Episode 987 || Avg rewards 76.0
Episode 988 || Avg rewards 110.0
Episode 989 || Avg rewards 60.0
Episode 990 || Avg rewards 46.0
Episode 991 || Avg rewards 86.0
Episode 992 || Avg rewards 43.0
Episode 993 || Avg rewards 54.0
Episode 994 || Avg rewards 41.0

Episode 995 || Avg rewards 48.0
Episode 996 || Avg rewards 65.0
Episode 997 || Avg rewards 47.0
Episode 998 || Avg rewards 57.0
Episode 999 || Avg rewards 64.0

3000 Episodes (Q Learning)

(Data started from episode 2003, as my device could not display the previous episodes due to sheet amount of raw data)

Episode 2003 || Avg rewards 59.0
Episode 2004 || Avg rewards 59.0
Episode 2005 || Avg rewards 57.0
Episode 2006 || Avg rewards 31.0
Episode 2007 || Avg rewards 30.0
Episode 2008 || Avg rewards 22.0
Episode 2009 || Avg rewards 63.0
Episode 2010 || Avg rewards 57.0
Episode 2011 || Avg rewards 35.0
Episode 2012 || Avg rewards 45.0
Episode 2013 || Avg rewards 98.0
Episode 2014 || Avg rewards 55.0
Episode 2015 || Avg rewards 20.0
Episode 2016 || Avg rewards 37.0
Episode 2017 || Avg rewards 42.0
Episode 2018 || Avg rewards 63.0
Episode 2019 || Avg rewards 47.0
Episode 2020 || Avg rewards 70.0
Episode 2021 || Avg rewards 39.0
Episode 2022 || Avg rewards 57.0
Episode 2023 || Avg rewards 108.0
Episode 2024 || Avg rewards 49.0
Episode 2025 || Avg rewards 53.0
Episode 2026 || Avg rewards 45.0
Episode 2027 || Avg rewards 55.0
Episode 2028 || Avg rewards 39.0
Episode 2029 || Avg rewards 49.0
Episode 2030 || Avg rewards 31.0
Episode 2031 || Avg rewards 49.0
Episode 2032 || Avg rewards 37.0
Episode 2033 || Avg rewards 65.0
Episode 2034 || Avg rewards 28.0

Episode 2035 || Avg rewards 69.0
Episode 2036 || Avg rewards 30.0
Episode 2037 || Avg rewards 73.0
Episode 2038 || Avg rewards 45.0
Episode 2039 || Avg rewards 55.0
Episode 2040 || Avg rewards 82.0
Episode 2041 || Avg rewards 37.0
Episode 2042 || Avg rewards 53.0
Episode 2043 || Avg rewards 93.0
Episode 2044 || Avg rewards 46.0
Episode 2045 || Avg rewards 98.0
Episode 2046 || Avg rewards 48.0
Episode 2047 || Avg rewards 41.0
Episode 2048 || Avg rewards 51.0
Episode 2049 || Avg rewards 37.0
Episode 2050 || Avg rewards 51.0
Episode 2051 || Avg rewards 45.0
Episode 2052 || Avg rewards 41.0
Episode 2053 || Avg rewards 37.0
Episode 2054 || Avg rewards 63.0
Episode 2055 || Avg rewards 43.0
Episode 2056 || Avg rewards 63.0
Episode 2057 || Avg rewards 50.0
Episode 2058 || Avg rewards 57.0
Episode 2059 || Avg rewards 31.0
Episode 2060 || Avg rewards 43.0
Episode 2061 || Avg rewards 39.0
Episode 2062 || Avg rewards 49.0
Episode 2063 || Avg rewards 55.0
Episode 2064 || Avg rewards 56.0
Episode 2065 || Avg rewards 82.0
Episode 2066 || Avg rewards 59.0
Episode 2067 || Avg rewards 39.0
Episode 2068 || Avg rewards 76.0
Episode 2069 || Avg rewards 90.0
Episode 2070 || Avg rewards 41.0
Episode 2071 || Avg rewards 26.0
Episode 2072 || Avg rewards 39.0
Episode 2073 || Avg rewards 105.0

.
. (Due to limited space I have curtailed the data to 5 pages for each trial)
.

Episode 2866 || Avg rewards 65.0
Episode 2867 || Avg rewards 43.0
Episode 2868 || Avg rewards 105.0
Episode 2869 || Avg rewards 57.0
Episode 2870 || Avg rewards 30.0
Episode 2871 || Avg rewards 34.0
Episode 2872 || Avg rewards 29.0
Episode 2873 || Avg rewards 20.0
Episode 2874 || Avg rewards 47.0
Episode 2875 || Avg rewards 59.0
Episode 2876 || Avg rewards 28.0
Episode 2877 || Avg rewards 41.0
Episode 2878 || Avg rewards 30.0
Episode 2879 || Avg rewards 32.0
Episode 2880 || Avg rewards 47.0
Episode 2881 || Avg rewards 39.0
Episode 2882 || Avg rewards 40.0
Episode 2883 || Avg rewards 49.0
Episode 2884 || Avg rewards 69.0
Episode 2885 || Avg rewards 43.0
Episode 2886 || Avg rewards 80.0
Episode 2887 || Avg rewards 25.0
Episode 2888 || Avg rewards 51.0
Episode 2889 || Avg rewards 51.0
Episode 2890 || Avg rewards 49.0
Episode 2891 || Avg rewards 52.0
Episode 2892 || Avg rewards 82.0
Episode 2893 || Avg rewards 63.0
Episode 2894 || Avg rewards 76.0
Episode 2895 || Avg rewards 43.0
Episode 2896 || Avg rewards 83.0
Episode 2897 || Avg rewards 41.0
Episode 2898 || Avg rewards 62.0
Episode 2899 || Avg rewards 38.0
Episode 2900 || Avg rewards 39.0
Episode 2901 || Avg rewards 74.0
Episode 2902 || Avg rewards 53.0
Episode 2903 || Avg rewards 20.0
Episode 2904 || Avg rewards 33.0
Episode 2905 || Avg rewards 28.0
Episode 2906 || Avg rewards 53.0
Episode 2907 || Avg rewards 60.0
Episode 2908 || Avg rewards 36.0

Episode 2909 || Avg rewards 53.0
Episode 2910 || Avg rewards 69.0
Episode 2911 || Avg rewards 35.0
Episode 2912 || Avg rewards 24.0
Episode 2913 || Avg rewards 59.0
Episode 2914 || Avg rewards 34.0
Episode 2915 || Avg rewards 61.0
Episode 2916 || Avg rewards 29.0
Episode 2917 || Avg rewards 49.0
Episode 2918 || Avg rewards 28.0
Episode 2919 || Avg rewards 51.0
Episode 2920 || Avg rewards 56.0
Episode 2921 || Avg rewards 61.0
Episode 2922 || Avg rewards 101.0
Episode 2923 || Avg rewards 100.0
Episode 2924 || Avg rewards 55.0
Episode 2925 || Avg rewards 57.0
Episode 2926 || Avg rewards 49.0
Episode 2927 || Avg rewards 47.0
Episode 2928 || Avg rewards 51.0
Episode 2929 || Avg rewards 55.0
Episode 2930 || Avg rewards 51.0
Episode 2931 || Avg rewards 86.0
Episode 2932 || Avg rewards 32.0
Episode 2933 || Avg rewards 43.0
Episode 2934 || Avg rewards 45.0
Episode 2935 || Avg rewards 65.0
Episode 2936 || Avg rewards 55.0
Episode 2937 || Avg rewards 49.0
Episode 2938 || Avg rewards 94.0
Episode 2939 || Avg rewards 24.0
Episode 2940 || Avg rewards 23.0
Episode 2941 || Avg rewards 41.0
Episode 2942 || Avg rewards 49.0
Episode 2943 || Avg rewards 52.0
Episode 2944 || Avg rewards 40.0
Episode 2945 || Avg rewards 34.0
Episode 2946 || Avg rewards 53.0
Episode 2947 || Avg rewards 47.0
Episode 2948 || Avg rewards 73.0
Episode 2949 || Avg rewards 35.0
Episode 2950 || Avg rewards 61.0
Episode 2951 || Avg rewards 33.0

Episode 2952 || Avg rewards 55.0
Episode 2953 || Avg rewards 55.0
Episode 2954 || Avg rewards 55.0
Episode 2955 || Avg rewards 82.0
Episode 2956 || Avg rewards 41.0
Episode 2957 || Avg rewards 57.0
Episode 2958 || Avg rewards 51.0
Episode 2959 || Avg rewards 37.0
Episode 2960 || Avg rewards 30.0
Episode 2961 || Avg rewards 51.0
Episode 2962 || Avg rewards 76.0
Episode 2963 || Avg rewards 84.0
Episode 2964 || Avg rewards 54.0
Episode 2965 || Avg rewards 53.0
Episode 2966 || Avg rewards 43.0
Episode 2967 || Avg rewards 48.0
Episode 2968 || Avg rewards 90.0
Episode 2969 || Avg rewards 61.0
Episode 2970 || Avg rewards 53.0
Episode 2971 || Avg rewards 33.0
Episode 2972 || Avg rewards 56.0
Episode 2973 || Avg rewards 28.0
Episode 2974 || Avg rewards 45.0
Episode 2975 || Avg rewards 33.0
Episode 2976 || Avg rewards 43.0
Episode 2977 || Avg rewards 51.0
Episode 2978 || Avg rewards 55.0
Episode 2979 || Avg rewards 27.0
Episode 2980 || Avg rewards 59.0
Episode 2981 || Avg rewards 28.0
Episode 2982 || Avg rewards 59.0
Episode 2983 || Avg rewards 43.0
Episode 2984 || Avg rewards 21.0
Episode 2985 || Avg rewards 43.0
Episode 2986 || Avg rewards 29.0
Episode 2987 || Avg rewards 88.0
Episode 2988 || Avg rewards 27.0
Episode 2989 || Avg rewards 28.0
Episode 2990 || Avg rewards 21.0
Episode 2991 || Avg rewards 53.0
Episode 2992 || Avg rewards 84.0
Episode 2993 || Avg rewards 36.0
Episode 2994 || Avg rewards 57.0

Episode 2995 || Avg rewards 92.0
Episode 2996 || Avg rewards 41.0
Episode 2997 || Avg rewards 84.0
Episode 2998 || Avg rewards 65.0
Episode 2999 || Avg rewards 37.0